İçindekiler

1 Projenin Özeti **2**

2 Projenin Amacı ve Kapsamı **2**

3 Genel Uygulama Akışı **2**

4 Proje Dizin Yapısı **2**

5 Projeyi Nasıl İlerlettik **3**

6 Kayıtların Saklanması **3**

7 Kullanıcı Deneyimi Tarafı **3**

8 Arayüz Tarafı **3**

9 Kod Bölümleri **5**

9.1 1. pubspec.yaml - Proje bağımlılıkları ve asset tanımları 6
9.2 2. lib/main.dart - Uygulamanın giriş noktası 7
9.3 3. lib/app/app.dart - Uygulama kurulumu ve Provider bağlama 8
9.4 4. lib/app/theme.dart - Tema, renk sistemi ve ortak görünüm kararları 9
9.5 5. lib/data/demo_content.dart - Demo otel ve oda verileri 12
9.6 6. lib/models/hotel_models.dart - Veri modelleri ve serileştirme mantığı 18
9.7 7. lib/state/hotel_app_state.dart - Merkezi uygulama durumu ve veri kalıcılığı 21
9.8 8. lib/screens/hotel_shell.dart - Ana kabuk, menü yapısı ve responsive navigasyon 25
9.9 9. lib/screens/hotels_screen.dart - Otel listeleme, arama ve filtreleme ekranı 30
9.10 10. lib/screens/booking_screen.dart - Rezervasyon formu, tarih seçimi ve fiyat özeti 39
9.11 11. lib/screens/reservations_screen.dart - Yaklaşan ve geçmiş rezervasyonları gösteren ekran . 58
9.12 12. lib/utils/formatters.dart - Para ve tarih biçimlendirme yardımcıları 61
9.13 13. lib/widgets/hotel_widgets.dart - Tekrar kullanılabilir ortak widget bileşenleri 62
9.14 14. test/widget_test.dart - Widget testleri ve temel senaryolar 67

10 Uygulama Ekran Görselleri **69**

Projenin Özeti

Bu projeyi üniversitedeki final projemiz olarak hazırladık. Amacımız, bir otel rezervasyon uygulamasının kullanıcı tarafında nasıl görüneceğini ve nasıl işleyeceğini daha anlaşılır bir şekilde ortaya koymaktı. Kullanıcı uygulamaya girdiğinde otellere bakabiliyor, oda seçebiliyor, tarih belirleyebiliyor ve rezervasyon sürecini baştan sona takip edebiliyor.

Projeyi geliştirirken sadece birkaç ekran göstermek istemedik. Daha çok gerçek bir uygulama hissi vermeye çalıştık. Bu yüzden önce arayüzü Figma üzerinde tasarladık, ekranların nasıl görüneceğini netleştirdik ve daha sonra bu tasarımı Flutter tarafına aktardık. Ardından uygulamanın gerçekten çalışması için butonlar, menü geçişleri ve veri düzeni gibi arka plan kısımlarını ekledik. Böylece ortaya hem göze daha yakın gelen hem de kullanım akışı daha derli toplu olan bir çalışma çıktı.

Projenin Amacı ve Kapsamı

Bu projede temel hedefimiz, otel rezervasyonu fikrini öğrenci projesi ölçeğinde anlaşılır ve düzenli bir şekilde ortaya koymaktı. Önce uygulamanın arayüz tarafını Figma üzerinde düşündük, ekranların nasıl görüneceğini planladık ve kullanıcı akışını kurduk. Daha sonra bu tasarımı Flutter içine taşıyarak çalışan bir mobil uygulama hâline getirdik.

İlerleyen aşamada uygulamanın sadece görünüşten ibaret kalmaması için arka planda çalışacak mantığı da ekledik. Yani önce kullanıcıya görünen bölümü hazırladık, sonra bu ekranların gerçekten çalışmasını sağlayan backend tarafını ekledik. Burada butonların işlev kazanması, menüler arası geçişlerin çalışması ve verilerin uygun şekilde ayarlanması gibi kısımlar sonradan yazıldı. Bu yüzden proje hem tasarım yönünü hem de uygulamanın işleyen tarafını birlikte anlatan bir final çalışmasına dönüştü.

Genel Uygulama Akışı

Uygulamanın genel akışını olabildiğince sade kurduk. Kullanıcı önce otelleri görüyor, ardından ilgisini çeken bir oteli seçiyor. Sonraki adımda oda seçimi yapıyor, kalacağı tarihleri belirliyor ve kendine uygun rezervasyon bilgilerini dolduruyor.

Biz burada kullanıcıyı gereksiz karmaşaya sokmamaya dikkat ettik. Adımlar birbirini doğal şekilde takip ediyor. Rezervasyon tamamlandıktan sonra da yapılan işlem kaybolmuyor; kullanıcı daha sonra kendi kayıtlarını tekrar görebiliyor. Bu da projeyi sadece ekran gösteren bir çalışma olmaktan çıkarıp daha tamamlanmış bir uygulama örneğine yaklaştırdı.

1. Kullanıcı oteller ekranına gelir ve listeyi inceler.
2. Arama kutusu ve filtre alt sayfası ile uygun tesisleri daraltır.
3. Bir otel kartına tıklayarak rezervasyon ekranına geçer.
4. Seçili otelin oda varyantları arasından uygun olanı belirler.
5. Tarih aralığını, misafir sayısını ve ek hizmetleri seçer.
6. Form alanlarını doldurup rezervasyonu oluşturur.
7. Kaydı rezervasyonlar ekranında yaklaşan veya geçmiş listesinde görür.

Proje Dizin Yapısı

Projeyi geliştirirken dosyaları da karışmayacak şekilde ayırmaya çalıştık. Böylece hangi kısmın ekranlarla, hangi kısmın veriyle ya da ortak bileşenlerle ilgili olduğu daha rahat anlaşılabilir oldu. Bu düzen bize rapor yazarken ve projenin hangi bölümde ne yaptığını anlatırken de kolaylık sağladı.

```
hotel_rezervasyon/├──
 assets/│
   └── images/│
       ├── deluxe.jpg│
       ├── standard.jpg│
       ├── suite.jpg│
```

```
        └── stock/
            ├── breakfast.jpg
            ├── lobby.jpg
            ├── pool.jpg
            └── room.jpg
lib/
    ├── app/
    ├── data/
    ├── models/
    ├── screens/
    ├── state/
    ├── utils/
    ├── widgets/
    └── main.dart
test/
    └── widget_test.dart
pubspec.yaml
```

Projeyi Nasıl İlerlettik

Projeyi geliştirirken önce kullanıcıda ilk izlenimi oluşturacak arayüzü hazırladık. Bu aşamada Figma üzerinde hangi sayfaların olacağına, kullanıcı nerede ne göreceğine ve rezervasyon sürecinin nasıl ilerleyeceğine karar verdik. Daha sonra bu taslağı Flutter içinde parça parça kurarak çalışan ekranlara dönüştürdük.

Arayüz oturduktan sonra uygulamanın arka tarafta gerçekten bir mantıkla çalışması için gerekli bölümleri ekledik. Yani kullanıcı bir tuşa bastığında doğru işlemin olması, menüler arasında geçişlerin düzgün çalışması, seçimlerden sonra uygun verilerin ekrana gelmesi ve rezervasyon tamamlandığında bunun kaybolmaması gibi backend tarafını ilgilendiren kısımlar sonradan geliştirilerek projeye eklendi.

Kayıtların Saklanması

Bizim için önemli olan noktalardan biri de kullanıcının yaptığı işlemin uygulama içinde kaybolmamasıydı. Bu yüzden otel seçimi, oda bilgisi ve oluşturulan rezervasyon gibi verilerin uygulama içinde düzenli şekilde ilerlemesine dikkat ettik.

Ayrıca yapılan rezervasyonların daha sonra tekrar görülebilmesi projeyi daha gerçekçi gösterdi. Kullanıcı uygulamayı kapatıp açsa bile önceki kayıtları yeniden görebiliyor. Bu ayrıntı, final projesi olarak sunarken uygulamanın daha tamamlanmış görünmesine katkı sağladı.

Kullanıcı Deneyimi Tarafı

Kullanıcı uygulamayı kullanırken zorlanmasın diye arama ve filtreleme kısımlarını ekledik. Böylece listede kaybolmadan daha uygun otellere daha kolay ulaşılabiliyor. Aynı şekilde rezervasyon formunda da kullanıcıdan istenen bilgiler olabildiğince anlaşılır şekilde verildi.

Burada amacımız teknik olarak karmaşık görünmekten çok, kullanıcı açısından akıcı bir deneyim sunmaktı. Yani kişi neyi, neden girdiğini anlayabilsin; uygulama da gerekli yerlerde onu yönlendirsin istedik. Bu yaklaşım projeyi daha insani ve daha kullanılabilir bir noktaya taşıdı.

Arayüz Tarafı

Arayüz tarafında sade ama düzenli bir görünüm oluşturmaya çalıştık. Çok ağır ya da karışık bir tasarım yerine, öğrencilerin geliştirdiği bir final projesine yakışacak şekilde temiz, anlaşılır ve sunumda rahat gösterilebilecek bir yapı kurduk. Bu görsel düzeni önce Figma üzerinde çizdik, sonra Flutter tarafında aynı hissi verecek şekilde uygulamaya aktardık.

Ekranların farklı boyutlarda da düzgün görünmesine dikkat ettik. Çünkü projeyi sadece kod olarak değil, görsel olarak da sunacağımız için uygulamanın farklı ekranlarda bozulmadan çalışması bizim için önemliydi. Bu yüzden arayüzü önce Figma tarafında netleştirip sonra Flutter içinde buna uygun şekilde düzenledik. Ardından arka tarafta buton işlevleri, geçişler ve veri hareketleri eklenince uygulama sadece güzel görünen değil, aynı zamanda çalışan bir yapıya dönüştü.

Kod Bölümleri

Aşağıdaki bölümde projeyi oluşturan temel dosyalar tek tek verilmiş, her dosyanın ne işe yaradığı anlatılmış ve ilgili kod içerikleri rapora eklenmiştir.

1. pubspec.yaml - Proje bağımlılıkları ve asset tanımları

Dosyanın görevi: Bu dosya Flutter projesinin paket bağımlılıklarını, SDK aralığını ve uygulamada kullanılacak görsel klasörlerini tanımlar.

Satır sayısı: 25

```
name: hotel_reservation
description: "A new Flutter project."
publish_to: 'none'
version: 1.0.0+1

environment:
  sdk: '>=3.2.3 <4.0.0'

dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.2
  provider: ^6.1.1
  shared_preferences: ^2.2.2

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^2.0.0

flutter:
  uses-material-design: true
  assets:
    - assets/images/
    - assets/images/stock/
```

Açıklama: Rapor açısından bu dosya önemlidir; çünkü projenin yalnızca görsel olarak değil, teknik olarak da hangi çerçeve ve paketler üstüne kurulduğunu burada görüyoruz. ‘provider‘ durum yönetimi için, ‘shared_preferences‘ ise rezervasyonları yerel bellekte tutmak için kullanılmıştır.

2. lib/main.dart - Uygulamanın giriş noktası

Dosyanın görevi: Bu dosya uygulamanın başlangıç dosyasıdır. ‘main()‘ fonksiyonu çalışır ve ‘MyApp‘ widget’ını ekrana basar.

Satır sayısı: 7

```
import 'package:flutter/material.dart';

import 'app/app.dart';

void main() {
  runApp(const MyApp());
}
```

Açıklama: Kod çok kısa tutulmuştur. Bu tercih iyi bir ayrım sağlar; başlangıç mantığı burada kalır, asıl uygulama kurulumu ise ‘app/app.dart‘ içine taşınır.

3. lib/app/app.dart - Uygulama kurulumu ve Provider bağlama

Dosyanın görevi: Bu dosya ‘MaterialApp‘ ağacını oluşturur, temayı uygular ve ‘HotelAppState‘ nesnesini ‘ChangeNotifierProvider‘ ile tüm ekranlara açar.

Satır sayısı: 23

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../screens/hotel_shell.dart';
import '../state/hotel_app_state.dart';
import 'theme.dart';

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (_) => HotelAppState()..load(),
      child: MaterialApp(
        title: 'StayBook',
        debugShowCheckedModeBanner: false,
        theme: buildAureaTheme(),
        home: const HotelShell(),
      ),
    );
  }
}
```

Açıklama: Buradaki yapı Flutter projelerinde sık görülen temiz bir kurulum örneğidir. State nesnesinin başlangıçta ‘load()‘ çağrısı ile hazırlanması, daha uygulama açılırken önceki rezervasyonların ekrana taşınmasına imkân verir.

4. lib/app/theme.dart - Tema, renk sistemi ve ortak görünüm kararları

Dosyanın görevi: Bu dosya uygulamanın bütün görsel dilini belirler. Renk paleti, yazı stilleri, kart biçimleri, buton temaları, input alanları ve navigasyon görünümü burada merkezi biçimde tanımlanır.

Satır sayısı: 156

```
import 'package:flutter/material.dart';

ThemeData buildAureaTheme() {
  const ink = Color(0xFF18324A);
  const coral = Color(0xFFF05A28);
  const teal = Color(0xFF0F8B7A);
  const mist = Color(0xFFF3F5F7);

  final scheme = ColorScheme.fromSeed(
    seedColor: ink,
    primary: ink,
    secondary: coral,
    tertiary: teal,
    surface: Colors.white,
    error: const Color(0xFF8C2F39),
    brightness: Brightness.light,
  );

  return ThemeData(
    useMaterial3: true,
    colorScheme: scheme,
    scaffoldBackgroundColor: mist,
    textTheme: const TextTheme(
      displayLarge: TextStyle(
        fontSize: 40,
        fontWeight: FontWeight.w700,
        letterSpacing: -0.9,
        color: ink,
      ),
      displayMedium: TextStyle(
        fontSize: 30,
        fontWeight: FontWeight.w700,
        letterSpacing: -0.5,
        color: ink,
      ),
      headlineMedium: TextStyle(
        fontSize: 24,
        fontWeight: FontWeight.w700,
        letterSpacing: -0.3,
        color: ink,
      ),
      titleLarge: TextStyle(
        fontSize: 18,
        fontWeight: FontWeight.w700,
        letterSpacing: -0.1,
        color: ink,
      ),
      titleMedium: TextStyle(
        fontSize: 15,
        fontWeight: FontWeight.w700,
        color: ink,
      ),
      bodyLarge: TextStyle(
        fontSize: 15,
        height: 1.45,
        color: Color(0xFF4A5B68),
      ),
      bodyMedium: TextStyle(
        fontSize: 13,
        height: 1.4,
```

```
        color: Color(0xFF5B6C79),
      ),
      labelLarge: TextStyle(
        fontSize: 13,
        fontWeight: FontWeight.w700,
      ),
    ),
    appBarTheme: const AppBarTheme(
      backgroundColor: Colors.transparent,
      foregroundColor: ink,
      elevation: 0,
      scrolledUnderElevation: 0,
    ),
    cardTheme: CardThemeData(
      color: Colors.white,
      elevation: 0,
      margin: EdgeInsets.zero,
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(20)),
    ),
    inputDecorationTheme: InputDecorationTheme(
      filled: true,
      fillColor: Colors.white,
      contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 15),
      border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(14),
        borderSide: BorderSide(color: ink.withValues(alpha: 0.08)),
      ),
      enabledBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(14),
        borderSide: BorderSide(color: ink.withValues(alpha: 0.08)),
      ),
      focusedBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(14),
        borderSide: BorderSide(color: coral, width: 1.5),
      ),
      hintStyle: const TextStyle(color: Color(0xFF7A8B8F)),
    ),
    filledButtonTheme: FilledButtonThemeData(
      style: FilledButton.styleFrom(
        backgroundColor: ink,
        foregroundColor: Colors.white,
        padding: const EdgeInsets.symmetric(horizontal: 18, vertical: 14),
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(14)),
        textStyle: const TextStyle(
          fontSize: 13,
          fontWeight: FontWeight.w700,
        ),
      ),
    ),
    outlinedButtonTheme: OutlinedButtonThemeData(
      style: OutlinedButton.styleFrom(
        foregroundColor: ink,
        side: BorderSide(color: ink.withValues(alpha: 0.14)),
        padding: const EdgeInsets.symmetric(horizontal: 18, vertical: 14),
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(14)),
        textStyle: const TextStyle(
          fontSize: 13,
          fontWeight: FontWeight.w700,
        ),
      ),
    ),
    chipTheme: ChipThemeData(
      backgroundColor: Colors.white,
      selectedColor: coral.withValues(alpha: 0.12),
      padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 8),
      labelStyle: const TextStyle(
```

```
        color: ink,
        fontWeight: FontWeight.w600,
        fontSize: 12,
      ),
      side: BorderSide(color: ink.withValues(alpha: 0.08)),
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(999)),
    ),
    navigationBarTheme: NavigationBarThemeData(
      backgroundColor: Colors.white,
      indicatorColor: coral.withValues(alpha: 0.14),
      labelTextStyle: WidgetStateProperty.resolveWith(
        (states) => TextStyle(
          color: states.contains(WidgetState.selected)
              ? ink
              : ink.withValues(alpha: 0.62),
          fontWeight: states.contains(WidgetState.selected)
              ? FontWeight.w700
              : FontWeight.w500,
        ),
      ),
    ),
    snackBarTheme: SnackBarThemeData(
      behavior: SnackBarBehavior.floating,
      backgroundColor: ink,
      contentTextStyle: const TextStyle(color: Colors.white),
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(14)),
    ),
    dividerColor: ink.withValues(alpha: 0.08),
  );
}
```

Açıklama: Tema dosyası ayrı tutulduğu için arayüz kararları tek noktadan kontrol edilebilir hâle gelmiştir. Bu, ders projesi için önemli bir artıdır; çünkü uygulama ileride büyüse bile farklı ekranlar arasında stil tutarlılığı korunur.

5. lib/data/demo_content.dart - Demo otel ve oda verileri

Dosyanın görevi: Bu dosya dış servis kullanılmadan çalışan örnek veri setini içerir. Otel listeleri, oda seçenekleri, etiketler, fiyatlar ve görsel yolları burada sabit veri olarak tanımlanmıştır.

Satır sayısı: 357

```
import 'package:flutter/material.dart';

import '../models/hotel_models.dart';

const List<HotelListing> demoHotels = [
  HotelListing(
    id: 'harbor_light',
    name: 'Liman şığıI Oteli',
    city: İ'stanbul',
    area: 'Karaköy',
    imagePath: 'assets/images/stock/lobby.jpg',
    rating: 4.8,
    reviewCount: 1284,
    priceFrom: 190,
    description: 'Merkezde, spa ve ıkahvalt ıodakl şehir oteli',
    tags: ['ıKahvalt', 'Spa', 'Teras'],
    accent: Color(0xFF2D6A73),
  ),
  HotelListing(
    id: 'bosphorus_palace',
    name: 'ğBoaz ıSaray',
    city: İ'stanbul',
    area: 'şşBeikta',
    imagePath: 'assets/images/stock/pool.jpg',
    rating: 4.9,
    reviewCount: 2031,
    priceFrom: 320,
    description: 'Transfer ve manzara ıodakl premium resort',
    tags: ['Transfer', 'Manzara', 'Havuz'],
    accent: Color(0xFFB96B47),
  ),
  HotelListing(
    id: 'moda_marina',
    name: 'Moda Marina Süitleri',
    city: İ'stanbul',
    area: 'ıKadköy',
    imagePath: 'assets/images/stock/room.jpg',
    rating: 4.7,
    reviewCount: 864,
    priceFrom: 240,
    description: İş' ıodakl butik konaklama ve ııhzl internet',
    tags: İş['', 'ııHzl Wi-Fi', 'Geç çıışk'],
    accent: Color(0xFF6C5B7B),
  ),
  HotelListing(
    id: 'galata_grand',
    name: 'Galata Grand Oteli',
    city: İ'stanbul',
    area: 'Galata',
    imagePath: 'assets/images/stock/breakfast.jpg',
    rating: 4.6,
    reviewCount: 693,
    priceFrom: 210,
    description: 'ıKahvalt ve şehir ımanzarasyla öne çıkan otel',
    tags: ['ıKahvalt', 'Manzara', 'Spor salonu'],
    accent: Color(0xFF475569),
  ),
  HotelListing(
    id: 'pera_nexus',
    name: 'Pera Nexus Oteli',
```

```
    city: İ'stanbul',
    area: 'Pera',
    imagePath: 'assets/images/stock/lobby.jpg',
    rating: 4.7,
    reviewCount: 918,
    priceFrom: 230,
    description: 'Spa, şi ve ıkahvalt segmenti',
    tags: ['Spa', İş'', 'ıKahvalt'],
    accent: Color(0xFF9A4D36),
  ),
  HotelListing(
    id: 'adalar_blue',
    name: 'Adalar Blue Resort',
    city: İ'stanbul',
    area: 'Büyükada',
    imagePath: 'assets/images/stock/pool.jpg',
    rating: 4.8,
    reviewCount: 1106,
    priceFrom: 280,
    description: 'Manzara, transfer ve sahil ıkonaklamas',
    tags: ['Manzara', 'Transfer', 'Spa'],
    accent: Color(0xFF2E5B88),
  ),
];

const List<RoomOffer> demoRooms = [
  RoomOffer(
    id: 'harbor_standard',
    hotelId: 'harbor_light',
    name: 'Standart Oda',
    collection: Ş'ehir ıKat',
    tagline: 'Merkezi konaklama',
    description: 'ıKsa şi seyahatleri ve hafta sonu kaçıamaklar için.',
    imagePath: 'assets/images/standard.jpg',
    nightlyRate: 190,
    maxGuests: 2,
    bedLabel: '1 queen',
    sizeLabel: '28 m²',
    perks: ['ıKahvalt', 'Wi-Fi', 'ııAkll TV'],
    accent: Color(0xFF2D6A73),
  ),
  RoomOffer(
    id: 'harbor_deluxe',
    hotelId: 'harbor_light',
    name: 'Delüks Oda',
    collection: 'Liman ıManzaras',
    tagline: 'Daha şgeni ve sakin',
    description: 'Üst kat, lounge şeriimi ve şgeni oturma ıalan.',
    imagePath: 'assets/images/deluxe.jpg',
    nightlyRate: 260,
    maxGuests: 3,
    bedLabel: '1 king',
    sizeLabel: '38 m²',
    perks: ['Lounge', 'Mini bar', 'ğYamur şduu'],
    accent: Color(0xFF2D6A73),
  ),
  RoomOffer(
    id: 'harbor_club',
    hotelId: 'harbor_light',
    name: 'Kulüp Oda',
    collection: 'Sessiz Kat',
    tagline: 'Kompakt premium konaklama',
    description: Ş'ehir merkezinde daha sessiz ve seçkin konaklama.',
    imagePath: 'assets/images/suite.jpg',
    nightlyRate: 310,
    maxGuests: 3,
```

```
    bedLabel: '1 king',
    sizeLabel: '41 m²',
    perks: ['Spa şeriimi', 'Teras', 'ııHzl şgiri'],
    accent: Color(0xFF2D6A73),
  ),
  RoomOffer(
    id: 'bosphorus_deluxe',
    hotelId: 'bosphorus_palace',
    name: 'Deniz Delüks',
    collection: 'ğBoaz ıKanad',
    tagline: 'Deniz ımanzaral kat',
    description: 'Havuz şeriimi ve balkonlu premium oda.',
    imagePath: 'assets/images/deluxe.jpg',
    nightlyRate: 320,
    maxGuests: 3,
    bedLabel: '1 king',
    sizeLabel: '42 m²',
    perks: ['Havuz', 'Balkon', 'Vale'],
    accent: Color(0xFFB96B47),
  ),
  RoomOffer(
    id: 'bosphorus_suite',
    hotelId: 'bosphorus_palace',
    name: 'Saray Süiti',
    collection: İ'mza Seri',
    tagline: 'Üst segment konaklama',
    description: 'Oturma ıalan, transfer ve ııhzl check-in ile.',
    imagePath: 'assets/images/suite.jpg',
    nightlyRate: 540,
    maxGuests: 4,
    bedLabel: '1 king + sofa',
    sizeLabel: '63 m²',
    perks: ['Transfer', 'Butler', 'Havuz şeriimi'],
    accent: Color(0xFFB96B47),
  ),
  RoomOffer(
    id: 'bosphorus_bosphorus_suite',
    hotelId: 'bosphorus_palace',
    name: 'ğBoaz Süiti',
    collection: 'Panorama Seri',
    tagline: 'Tam cephe manzara',
    description: 'ğıBoaz gören şgeni oturma ıalanl üst segment oda.',
    imagePath: 'assets/images/standard.jpg',
    nightlyRate: 420,
    maxGuests: 3,
    bedLabel: '1 king',
    sizeLabel: '48 m²',
    perks: ['Manzara', 'Transfer', 'Özel şgiri'],
    accent: Color(0xFFB96B47),
  ),
  RoomOffer(
    id: 'moda_studio',
    hotelId: 'moda_marina',
    name: 'Marina Stüdyo',
    collection: 'Çışalma ıKat',
    tagline: 'ııHzl şehir şıyaam için',
    description: 'Çışalma ımasas ve ııhzl internet ıodakl düzen.',
    imagePath: 'assets/images/standard.jpg',
    nightlyRate: 240,
    maxGuests: 2,
    bedLabel: '1 queen',
    sizeLabel: '30 m²',
    perks: ['Çışalma ımasas', 'Wi-Fi', 'Kahve seti'],
    accent: Color(0xFF6C5B7B),
  ),
  RoomOffer(
```

```
    id: 'moda_suite',
    hotelId: 'moda_marina',
    name: 'Marina Süiti',
    collection: 'Yönetici Seri',
    tagline: 'Uzun konaklama için',
    description: 'ıAyr oturma bölümü ve geç çıışk ıavantaj.',
    imagePath: 'assets/images/suite.jpg',
    nightlyRate: 390,
    maxGuests: 4,
    bedLabel: '1 king + lounge',
    sizeLabel: '54 m²',
    perks: ['Geç çıışk', 'Mini mutfak', 'ıToplant ımasas'],
    accent: Color(0xFF6C5B7B),
  ),
  RoomOffer(
    id: 'moda_loft',
    hotelId: 'moda_marina',
    name: 'Marina Loft',
    collection: İş' Plus',
    tagline: 'Kompakt şi süiti',
    description: 'ıırHzl ıtoplantlar ve ıksa konaklamalar için ideal.',
    imagePath: 'assets/images/deluxe.jpg',
    nightlyRate: 330,
    maxGuests: 3,
    bedLabel: '1 king',
    sizeLabel: '37 m²',
    perks: İş[' ımasas', 'ıırHzl Wi-Fi', 'Geç çıışk'],
    accent: Color(0xFF6C5B7B),
  ),
  RoomOffer(
    id: 'galata_comfort',
    hotelId: 'galata_grand',
    name: 'Konfor Oda',
    collection: 'Grand Kat',
    tagline: 'ıKahvalt ıodakl konaklama',
    description: 'Güne erken şbalayan misafirler için pratik oda.',
    imagePath: 'assets/images/standard.jpg',
    nightlyRate: 210,
    maxGuests: 2,
    bedLabel: '1 queen',
    sizeLabel: '27 m²',
    perks: ['ıKahvalt salonu', 'Spor salonu', 'ıırHzl Wi-Fi'],
    accent: Color(0xFF475569),
  ),
  RoomOffer(
    id: 'galata_family',
    hotelId: 'galata_grand',
    name: 'Aile Delüks',
    collection: 'Köşe Seri',
    tagline: 'Daha şgeni aile ıodas',
    description: 'Aile veya uzun konaklamalar için şgeni çözüm.',
    imagePath: 'assets/images/deluxe.jpg',
    nightlyRate: 330,
    maxGuests: 4,
    bedLabel: '1 king + 2 sofa',
    sizeLabel: '46 m²',
    perks: ['ıKahvalt', 'Geç çıışk', 'Teras şeriimi'],
    accent: Color(0xFF475569),
  ),
  RoomOffer(
    id: 'galata_terrace',
    hotelId: 'galata_grand',
    name: 'Teras Süiti',
    collection: 'Panorama Kat',
    tagline: Ş'ehir ımanzaral üst segment',
    description: 'Galata çevresine bakan şgeni ıterasl oda.',
```

```
    imagePath: 'assets/images/suite.jpg',
    nightlyRate: 410,
    maxGuests: 3,
    bedLabel: '1 king',
    sizeLabel: '44 m²',
    perks: ['Manzara', 'ıKahvalt', 'Teras'],
    accent: Color(0xFF475569),
  ),
  RoomOffer(
    id: 'pera_compact',
    hotelId: 'pera_nexus',
    name: 'Pera Compact',
    collection: Ş'ehir Seri',
    tagline: 'ıKsa konaklama',
    description: 'Merkezi, ııhzl ve kompakt şehir ıodas.',
    imagePath: 'assets/images/standard.jpg',
    nightlyRate: 230,
    maxGuests: 2,
    bedLabel: '1 queen',
    sizeLabel: '24 m²',
    perks: ['ıKahvalt', 'Spa', 'ııHzl Wi-Fi'],
    accent: Color(0xFF9A4D36),
  ),
  RoomOffer(
    id: 'pera_corner',
    hotelId: 'pera_nexus',
    name: 'Pera Corner',
    collection: 'Köşe Seri',
    tagline: 'Daha şgeni şehir ıkat',
    description: 'Köşe cepheli, daha ııaydnlk ve sakin oda.',
    imagePath: 'assets/images/deluxe.jpg',
    nightlyRate: 320,
    maxGuests: 3,
    bedLabel: '1 king',
    sizeLabel: '35 m²',
    perks: ['Spa', 'ıKahvalt', İş' ımasas'],
    accent: Color(0xFF9A4D36),
  ),
  RoomOffer(
    id: 'pera_executive',
    hotelId: 'pera_nexus',
    name: 'Pera Executive',
    collection: 'Yönetici ıKat',
    tagline: İş' ıodakl üst segment',
    description: 'ıToplant, dinlenme ve ııhzl şehir ışıak için.',
    imagePath: 'assets/images/suite.jpg',
    nightlyRate: 410,
    maxGuests: 3,
    bedLabel: '1 king',
    sizeLabel: '45 m²',
    perks: İş[' salonu', 'Spa', 'Transfer'],
    accent: Color(0xFF9A4D36),
  ),
  RoomOffer(
    id: 'adalar_garden',
    hotelId: 'adalar_blue',
    name: 'Ada Bahçe Oda',
    collection: 'Bahçe ıKat',
    tagline: 'Sahil konforu',
    description: 'Bahçe şeriimli kompakt resort ıodas.',
    imagePath: 'assets/images/standard.jpg',
    nightlyRate: 280,
    maxGuests: 2,
    bedLabel: '1 queen',
    sizeLabel: '29 m²',
    perks: ['Transfer', 'ıKahvalt', 'Spa'],
```

```
    accent: Color(0xFF2E5B88),
  ),
  RoomOffer(
    id: 'adalar_panorama',
    hotelId: 'adalar_blue',
    name: 'Ada Panorama',
    collection: 'Deniz Seri',
    tagline: 'Tam cephe manzara',
    description: 'Denize bakan orta segment resort ıodas.',
    imagePath: 'assets/images/deluxe.jpg',
    nightlyRate: 360,
    maxGuests: 3,
    bedLabel: '1 king',
    sizeLabel: '39 m²',
    perks: ['Manzara', 'Transfer', 'Balkon'],
    accent: Color(0xFF2E5B88),
  ),
  RoomOffer(
    id: 'adalar_villa',
    hotelId: 'adalar_blue',
    name: 'Ada Villa Süiti',
    collection: 'Özel Seri',
    tagline: 'şGeni resort ıkonaklamas',
    description: 'Daha özel, daha şgeni ve resort ıodakl çözüm.',
    imagePath: 'assets/images/suite.jpg',
    nightlyRate: 490,
    maxGuests: 4,
    bedLabel: '1 king + sofa',
    sizeLabel: '58 m²',
    perks: ['Spa', 'Transfer', 'Manzara'],
    accent: Color(0xFF2E5B88),
  ),
];
```

Açıklama: Gerçek bir arka uç kullanılmadığı için ders kapsamında ihtiyaç duyulan veri akışı bu sabit listeler üzerinden çözülmüştür. Böylece arayüz, filtreleme ve rezervasyon mantığı veri kaynağına bağlı kalmadan gösterilebilir.

6. lib/models/hotel_models.dart - Veri modelleri ve serileştirme mantığı

Dosyanın görevi: Bu dosyada otel, oda ve rezervasyon için kullanılan veri sınıfları yer alır. ‘Reservation‘ modeli ayrıca JSON dönüşümü, gece hesabı ve toplam ücret hesabı gibi iş kurallarını da taşır.

Satır sayısı: 185

```
import 'package:flutter/material.dart';

class HotelListing {
  const HotelListing({
    required this.id,
    required this.name,
    required this.city,
    required this.area,
    required this.imagePath,
    required this.rating,
    required this.reviewCount,
    required this.priceFrom,
    required this.description,
    required this.tags,
    required this.accent,
  });

  final String id;
  final String name;
  final String city;
  final String area;
  final String imagePath;
  final double rating;
  final int reviewCount;
  final int priceFrom;
  final String description;
  final List<String> tags;
  final Color accent;
}

class RoomOffer {
  const RoomOffer({
    required this.id,
    required this.hotelId,
    required this.name,
    required this.collection,
    required this.tagline,
    required this.description,
    required this.imagePath,
    required this.nightlyRate,
    required this.maxGuests,
    required this.bedLabel,
    required this.sizeLabel,
    required this.perks,
    required this.accent,
  });

  final String id;
  final String hotelId;
  final String name;
  final String collection;
  final String tagline;
  final String description;
  final String imagePath;
  final int nightlyRate;
  final int maxGuests;
  final String bedLabel;
  final String sizeLabel;
  final List<String> perks;
  final Color accent;
```

```
}

class Reservation {
  const Reservation({
    required this.guestName,
    required this.email,
    required this.hotelId,
    required this.hotelName,
    required this.roomId,
    required this.roomName,
    required this.nightlyRate,
    required this.checkIn,
    required this.checkOut,
    required this.guests,
    required this.specialRequest,
    required this.extras,
    required this.extrasTotal,
    required this.createdAt,
  });

  final String guestName;
  final String email;
  final String hotelId;
  final String hotelName;
  final String roomId;
  final String roomName;
  final int nightlyRate;
  final DateTime checkIn;
  final DateTime checkOut;
  final int guests;
  final String specialRequest;
  final List<String> extras;
  final int extrasTotal;
  final DateTime createdAt;

  int get nights {
    final difference = checkOut.difference(checkIn).inDays;
    return difference > 0 ? difference : 1;
  }

  int get totalPrice => (nightlyRate * nights) + extrasTotal;

  Map<String, dynamic> toJson() {
    return {
      'guestName': guestName,
      'email': email,
      'hotelId': hotelId,
      'hotelName': hotelName,
      'roomId': roomId,
      'roomName': roomName,
      'nightlyRate': nightlyRate,
      'checkIn': checkIn.toIso8601String(),
      'checkOut': checkOut.toIso8601String(),
      'guests': guests,
      'specialRequest': specialRequest,
      'extras': extras,
      'extrasTotal': extrasTotal,
      'createdAt': createdAt.toIso8601String(),
    };
  }

  factory Reservation.fromJson(Map<String, dynamic> json) {
    final legacyRoomName =
        (json['roomName'] ?? json['room'] ?? 'Standart Oda').toString();
    final fallbackCheckIn = DateTime.now();
    final checkIn = _readDate(json['checkIn'], fallbackCheckIn);
```

```
    final checkOut = _readDate(
      json['checkOut'],
      fallbackCheckIn.add(const Duration(days: 1)),
    );

    return Reservation(
      guestName: (json['guestName'] ?? json['name'] ?? '').toString(),
      email: (json['email'] ?? 'guest@staybook.demo').toString(),
      hotelId: (json['hotelId'] ?? 'harbor_light').toString(),
      hotelName: (json['hotelName'] ?? json['hotel'] ?? 'Liman şığıI Oteli')
          .toString(),
      roomId: (json['roomId'] ?? _legacyRoomId(legacyRoomName)).toString(),
      roomName: legacyRoomName,
      nightlyRate: _readGuests(json['nightlyRate']),
      checkIn: checkIn,
      checkOut: checkOut,
      guests: _readGuests(json['guests']),
      specialRequest: (json['specialRequest'] ?? '').toString(),
      extras: _readExtras(json['extras']),
      extrasTotal: _readGuests(json['extrasTotal']),
      createdAt: _readDate(json['createdAt'], DateTime.now()),
    );
  }

  static DateTime _readDate(dynamic value, DateTime fallback) {
    if (value is String) {
      return DateTime.tryParse(value) ?? fallback;
    }
    if (value is int) {
      return DateTime.fromMillisecondsSinceEpoch(value);
    }
    return fallback;
  }

  static int _readGuests(dynamic value) {
    if (value is int) {
      return value;
    }
    return int.tryParse(value?.toString() ?? '') ?? 0;
  }

  static List<String> _readExtras(dynamic value) {
    if (value is List) {
      return value.map((item) => item.toString()).toList();
    }
    return const [];
  }

  static String _legacyRoomId(String roomName) {
    final value = roomName.toLowerCase();
    if (value.contains('suite') || value.contains('süit')) {
      return 'bosphorus_suite';
    }
    if (value.contains('deluxe') || value.contains('delüks')) {
      return 'harbor_deluxe';
    }
    return 'harbor_standard';
  }
}
```

Açıklama: Model katmanının ayrı tutulması kodun okunmasını kolaylaştırır. Özellikle ‘fromJson()‘ içindeki eski veri biçimleriyle uyumluluk sağlayan yapı, yerel depolamadaki önceki kayıtların boşa gitmemesi için düşünülmüş pratik bir çözümdür.

7. lib/state/hotel_app_state.dart - Merkezi uygulama durumu ve veri kalıcılığı

Dosyanın görevi: Bu dosya seçili oteli, seçili odayı, rezervasyon listesini ve yükleme kaydetme işlemlerini yönetir. Uygulamanın mantıksal merkezi burasıdır.

Satır sayısı: 206

```
import 'dart:convert';

import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

import '../data/demo_content.dart';
import '../models/hotel_models.dart';

class HotelAppState extends ChangeNotifier {
  static const _storageKey = 'staybook_reservations_v3';
  static const _legacyStorageKey = 'aurea_stay_reservations_v2';

  final List<HotelListing> hotels = List.unmodifiable(demoHotels);
  final List<RoomOffer> rooms = List.unmodifiable(demoRooms);

  String? _selectedHotelId;
  String? _selectedRoomId;
  List<Reservation> _reservations = [];

  String? get selectedHotelId => _selectedHotelId;

  HotelListing? get selectedHotel {
    final id = _selectedHotelId;
    if (id == null) {
      return null;
    }
    for (final hotel in hotels) {
      if (hotel.id == id) {
        return hotel;
      }
    }
    return null;
  }

  RoomOffer? get selectedRoom {
    final id = _selectedRoomId;
    if (id == null) {
      return null;
    }
    return roomForId(id);
  }

  List<RoomOffer> get roomsForSelectedHotel => List.unmodifiable(
        rooms
            .where((room) => room.hotelId == _selectedHotelId)
            .toList(growable: false),
      );

  List<Reservation> get reservations => List.unmodifiable(_reservations);

  List<Reservation> get upcomingReservations {
    final today = DateUtils.dateOnly(DateTime.now());
    final items = _reservations
        .where((reservation) =>
            !DateUtils.dateOnly(reservation.checkOut).isBefore(today))
        .toList()
      ..sort((a, b) => a.checkIn.compareTo(b.checkIn));
    return List.unmodifiable(items);
  }

```

```
  List<Reservation> get pastReservations {
    final today = DateUtils.dateOnly(DateTime.now());
    final items = _reservations
        .where((reservation) =>
            DateUtils.dateOnly(reservation.checkOut).isBefore(today))
        .toList()
      ..sort((a, b) => b.checkIn.compareTo(a.checkIn));
    return List.unmodifiable(items);
  }

  void selectHotel(String hotelId) {
    _selectedHotelId = hotelId;
    final hotelRooms = rooms.where((room) => room.hotelId == hotelId).toList();
    if (hotelRooms.isEmpty) {
      _selectedRoomId = null;
    } else if (!hotelRooms.any((room) => room.id == _selectedRoomId)) {
      _selectedRoomId = hotelRooms.first.id;
    }
    notifyListeners();
  }

  void selectRoom(String roomId) {
    final room = roomForId(roomId);
    if (room == null) {
      return;
    }

    _selectedRoomId = roomId;
    _selectedHotelId = room.hotelId;
    notifyListeners();
  }

  HotelListing hotelForId(String id) {
    return hotels.firstWhere(
      (hotel) => hotel.id == id,
      orElse: () => hotels.first,
    );
  }

  RoomOffer? roomForId(String id) {
    for (final room in rooms) {
      if (room.id == id) {
        return room;
      }
    }
    return null;
  }

  Future<void> load() async {
    final prefs = await SharedPreferences.getInstance();
    final raw = prefs.getString(_storageKey) ??
        prefs.getString(_legacyStorageKey) ??
        prefs.getString('reservations');

    if (raw == null || raw.isEmpty) {
      return;
    }

    try {
      final decoded = jsonDecode(raw);
      if (decoded is List) {
        _reservations = decoded
            .whereType<Map>()
            .map((entry) =>
                Reservation.fromJson(Map<String, dynamic>.from(entry)))
            .map(_hydrateLegacyReservation)
```

```
            .toList()
          ..sort((a, b) => b.createdAt.compareTo(a.createdAt));
        notifyListeners();
      }
    } catch (_) {
      _reservations = [];
    }
  }

  Future<void> saveReservation({
    required String guestName,
    required String email,
    required DateTime checkIn,
    required DateTime checkOut,
    required int guests,
    required String specialRequest,
    required List<String> extras,
    required int extrasTotal,
  }) async {
    final hotel = selectedHotel;
    final room = selectedRoom;
    if (hotel == null || room == null) {
      return;
    }

    final reservation = Reservation(
      guestName: guestName,
      email: email,
      hotelId: hotel.id,
      hotelName: hotel.name,
      roomId: room.id,
      roomName: room.name,
      nightlyRate: room.nightlyRate,
      checkIn: checkIn,
      checkOut: checkOut,
      guests: guests,
      specialRequest: specialRequest,
      extras: extras,
      extrasTotal: extrasTotal,
      createdAt: DateTime.now(),
    );

    _reservations = [reservation, ..._reservations];
    notifyListeners();

    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(
      _storageKey,
      jsonEncode(_reservations.map((item) => item.toJson()).toList()),
    );
  }

  Reservation _hydrateLegacyReservation(Reservation reservation) {
    final room = roomForId(reservation.roomId);
    if (room == null) {
      return reservation;
    }

    final hotel = hotelForId(room.hotelId);
    return Reservation(
      guestName: reservation.guestName,
      email: reservation.email,
      hotelId: reservation.hotelId.isEmpty ? hotel.id : reservation.hotelId,
      hotelName:
          reservation.hotelName.isEmpty ? hotel.name : reservation.hotelName,
      roomId: reservation.roomId,
```

```
      roomName: reservation.roomName,
      nightlyRate: reservation.nightlyRate == 0
          ? room.nightlyRate
          : reservation.nightlyRate,
      checkIn: reservation.checkIn,
      checkOut: reservation.checkOut,
      guests: reservation.guests == 0 ? 1 : reservation.guests,
      specialRequest: reservation.specialRequest,
      extras: reservation.extras,
      extrasTotal: reservation.extrasTotal,
      createdAt: reservation.createdAt,
    );
  }
}
```

Açıklama: ‘ChangeNotifier‘ yapısı sayesinde arayüz tarafı, state değiştiğinde kendini günceller. Ayrıca ‘SharedPreferences‘ ile kayıtların saklanması, uygulamayı kapatıp açınca rezervasyonların yeniden gösterilmesini sağlar.

8. lib/screens/hotel_shell.dart - Ana kabuk, menü yapısı ve responsive navigasyon

Dosyanın görevi: Bu ekran uygulamanın dış kabuğunu oluşturur. Geniş ekranda sol menü, dar ekranda alt gezinme çubuğu kullanır ve ilgili iç ekranı gösterir.

Satır sayısı: 269

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../state/hotel_app_state.dart';
import 'booking_screen.dart';
import 'hotels_screen.dart';
import 'reservations_screen.dart';

class HotelShell extends StatefulWidget {
  const HotelShell({super.key});

  @override
  State<HotelShell> createState() => _HotelShellState();
}

class _HotelShellState extends State<HotelShell> {
  int _currentIndex = 0;

  final List<_ShellItem> _items = const [
    _ShellItem('Oteller', Icons.apartment_rounded),
    _ShellItem('Rezervasyon', Icons.bed_rounded),
    _ShellItem('Rezervasyonlar', Icons.event_note_rounded),
  ];

  void _navigateTo(int index) {
    setState(() {
      _currentIndex = index;
    });
  }

  @override
  Widget build(BuildContext context) {
    final isWide = MediaQuery.sizeOf(context).width >= 1080;
    final body = AnimatedSwitcher(
      duration: const Duration(milliseconds: 220),
      child: KeyedSubtree(
        key: ValueKey(_currentIndex),
        child: _buildScreen(_currentIndex),
      ),
    );

    if (isWide) {
      return Scaffold(
        body: SafeArea(
          child: Row(
            children: [
              _Sidebar(
                items: _items,
                currentIndex: _currentIndex,
                onNavigate: _navigateTo,
              ),
              Expanded(
                child: Padding(
                  padding: const EdgeInsets.fromLTRB(0, 12, 12, 12),
                  child: body,
                ),
              ),
            ],
          ),
        ),
```

```
      );
    }

    return Scaffold(
      body: SafeArea(child: body),
      bottomNavigationBar: NavigationBar(
        selectedIndex: _currentIndex,
        onDestinationSelected: _navigateTo,
        destinations: _items
            .map(
              (item) => NavigationDestination(
                icon: Icon(item.icon),
                label: item.label,
              ),
            )
            .toList(),
      ),
    );
  }

  Widget _buildScreen(int index) {
    switch (index) {
      case 0:
        return HotelsScreen(onNavigate: _navigateTo);
      case 1:
        return const BookingScreen();
      case 2:
        return const ReservationsScreen();
      default:
        return HotelsScreen(onNavigate: _navigateTo);
    }
  }
}

class _Sidebar extends StatelessWidget {
  const _Sidebar({
    required this.items,
    required this.currentIndex,
    required this.onNavigate,
  });

  final List<_ShellItem> items;
  final int currentIndex;
  final ValueChanged<int> onNavigate;

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    return Container(
      width: 264,
      margin: const EdgeInsets.all(12),
      padding: const EdgeInsets.all(14),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(28),
        border: Border.all(
          color: theme.colorScheme.primary.withValues(alpha: 0.08),
        ),
        boxShadow: [
          BoxShadow(
            color: theme.colorScheme.primary.withValues(alpha: 0.035),
            blurRadius: 18,
            offset: const Offset(0, 8),
          ),
        ],
      ),
```

```
      child: Consumer<HotelAppState>(
        builder: (context, state, _) {
          return Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Container(
                padding:
                    const EdgeInsets.symmetric(horizontal: 12, vertical: 8),
                decoration: BoxDecoration(
                  color: theme.colorScheme.secondary,
                  borderRadius: BorderRadius.circular(999),
                ),
                child: Text(
                  'StayBook',
                  style: theme.textTheme.labelLarge?.copyWith(
                    color: Colors.white,
                  ),
                ),
              ),
              const SizedBox(height: 14),
              Text(
                'Otel rezervasyonu',
                style: theme.textTheme.headlineMedium,
              ),
              const SizedBox(height: 4),
              Text(
                'Kompakt ve ticari rezervasyon ışıak',
                style: theme.textTheme.bodyMedium,
              ),
              const SizedBox(height: 18),
              Container(
                padding: const EdgeInsets.all(16),
                decoration: BoxDecoration(
                  color: theme.colorScheme.primary.withValues(alpha: 0.05),
                  borderRadius: BorderRadius.circular(20),
                  border: Border.all(
                    color: theme.colorScheme.primary.withValues(alpha: 0.08),
                  ),
                ),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      state.selectedHotel?.name ?? 'Otel seçilmedi',
                      style: theme.textTheme.titleLarge,
                    ),
                    const SizedBox(height: 6),
                    Text(
                      state.selectedHotel == null
                          ? 'Oteller menüsünden bir tesis seçin'
                          : '${state.selectedHotel!.area}, ${state.selectedHotel!.city}',
                      style: theme.textTheme.bodyMedium,
                    ),
                    const SizedBox(height: 14),
                    Row(
                      children: [
                        Icon(
                          Icons.event_note_rounded,
                          size: 18,
                          color: theme.colorScheme.secondary,
                        ),
                        const SizedBox(width: 8),
                        Expanded(
                          child: Text(
                            '${state.upcomingReservations.length} şyaklaan rezervasyon',
                            style: theme.textTheme.titleMedium,
```

```
                      ),
                    ),
                  ],
                ),
              ],
            ),
          ),
          const SizedBox(height: 18),
          Expanded(
            child: ListView.separated(
              itemCount: items.length,
              separatorBuilder: (_, __) => const SizedBox(height: 8),
              itemBuilder: (context, index) {
                final item = items[index];
                final selected = currentIndex == index;
                return InkWell(
                  borderRadius: BorderRadius.circular(16),
                  onTap: () => onNavigate(index),
                  child: AnimatedContainer(
                    duration: const Duration(milliseconds: 180),
                    padding: const EdgeInsets.symmetric(
                      horizontal: 14,
                      vertical: 12,
                    ),
                    decoration: BoxDecoration(
                      color: selected
                          ? theme.colorScheme.secondary
                              .withValues(alpha: 0.1)
                          : Colors.transparent,
                      borderRadius: BorderRadius.circular(16),
                      border: Border.all(
                        color: selected
                            ? theme.colorScheme.secondary
                                .withValues(alpha: 0.18)
                            : Colors.transparent,
                      ),
                    ),
                    child: Row(
                      children: [
                        Icon(
                          item.icon,
                          color: selected
                              ? theme.colorScheme.secondary
                              : theme.colorScheme.primary
                                  .withValues(alpha: 0.62),
                        ),
                        const SizedBox(width: 10),
                        Text(
                          item.label,
                          style: theme.textTheme.titleMedium?.copyWith(
                            color: selected
                                ? theme.colorScheme.primary
                                : theme.colorScheme.primary
                                    .withValues(alpha: 0.82),
                          ),
                        ),
                      ],
                    ),
                  ),
                );
              },
            ),
          ),
        ],
      );
    },
```

```
      ),
    );
  }
}

class _ShellItem {
  const _ShellItem(this.label, this.icon);

  final String label;
  final IconData icon;
}
```

Açıklama: Buradaki yapı mobil ve geniş ekran uyumunu göstermek açısından değerlidir. ‘AnimatedSwitcher‘ ile geçişlerin yumuşatılması ve seçili otel bilgisinin menü kartında özetlenmesi kullanıcı deneyimini artırır.

9. lib/screens/hotels_screen.dart - Otel listeleme, arama ve filtreleme ekranı

Dosyanın görevi: Bu dosya otellerin kullanıcıya listelendiği ana ekranı kurar. Arama kutusu, puan filtresi, semt filtresi, özellik seçimi ve otel kartları burada oluşturulur.

Satır sayısı: 563

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/hotel_models.dart';
import '../state/hotel_app_state.dart';
import '../widgets/hotel_widgets.dart';

class HotelsScreen extends StatefulWidget {
  const HotelsScreen({
    super.key,
    required this.onNavigate,
  });

  final ValueChanged<int> onNavigate;

  @override
  State<HotelsScreen> createState() => _HotelsScreenState();
}

class _HotelsScreenState extends State<HotelsScreen> {
  final _searchController = TextEditingController();
  String? _selectedArea;
  double? _minimumRating;
  final Set<String> _selectedFeatures = {};

  @override
  void dispose() {
    _searchController.dispose();
    super.dispose();
  }

  List<HotelListing> _visibleHotels(HotelAppState state) {
    final query = _searchController.text.trim().toLowerCase();

    return state.hotels.where((hotel) {
      final matchesArea = _selectedArea == null || hotel.area == _selectedArea;
      final matchesRating =
          _minimumRating == null || hotel.rating >= _minimumRating!;
      final matchesFeatures = _selectedFeatures.every(hotel.tags.contains);

      if (!matchesArea || !matchesRating || !matchesFeatures) {
        return false;
      }

      if (query.isEmpty) {
        return true;
      }

      final searchText = [
        hotel.name,
        hotel.city,
        hotel.area,
        hotel.description,
        ...hotel.tags,
      ].join(' ').toLowerCase();

      return searchText.contains(query);
    }).toList(growable: false);
  }

```

```
  int get _activeFilterCount {
    var count = _selectedFeatures.length;
    if (_selectedArea != null) {
      count + = 1;
    }
    if (_minimumRating != null) {
      count + = 1;
    }
    return count;
  }

  Future<void> _openFilterSheet(HotelAppState state) async {
    final areas = state.hotels.map((hotel) => hotel.area).toSet().toList()
      ..sort();
    final features = state.hotels.expand((hotel) => hotel.tags).toSet().toList()
      ..sort();

    final result = await showModalBottomSheet<_FilterSelection>(
      context: context,
      isScrollControlled: true,
      showDragHandle: true,
      backgroundColor: Colors.transparent,
      builder: (context) {
        var tempArea = _selectedArea;
        var tempRating = _minimumRating;
        final tempFeatures = Set<String>.from(_selectedFeatures);

        return StatefulBuilder(
          builder: (context, setModalState) {
            return Container(
              decoration: const BoxDecoration(
                color: Colors.white,
                borderRadius: BorderRadius.vertical(top: Radius.circular(24)),
              ),
              child: SafeArea(
                top: false,
                child: Padding(
                  padding: const EdgeInsets.fromLTRB(18, 8, 18, 18),
                  child: SingleChildScrollView(
                    child: Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      mainAxisSize: MainAxisSize.min,
                      children: [
                        Row(
                          children: [
                            Expanded(
                              child: Text(
                                'Filtreler',
                                style:
                                    Theme.of(context).textTheme.headlineMedium,
                              ),
                            ),
                            TextButton(
                              onPressed: () {
                                setModalState(() {
                                  tempArea = null;
                                  tempRating = null;
                                  tempFeatures.clear();
                                });
                              },
                              child: const Text('Temizle'),
                            ),
                          ],
                        ),
                        const SizedBox(height: 14),
                        Text(
```

```
                  'Semt',
                  style: Theme.of(context).textTheme.titleMedium,
                ),
                const SizedBox(height: 10),
                Wrap(
                  spacing: 8,
                  runSpacing: 8,
                  children: areas
                      .map(
                        (area) => ChoiceChip(
                          label: Text(area),
                          selected: tempArea == area,
                          onSelected: (_) {
                            setModalState(() {
                              tempArea = tempArea == area ? null : area;
                            });
                          },
                        ),
                      )
                      .toList(),
                ),
                const SizedBox(height: 16),
                Text(
                  'Puan',
                  style: Theme.of(context).textTheme.titleMedium,
                ),
                const SizedBox(height: 10),
                Wrap(
                  spacing: 8,
                  runSpacing: 8,
                  children: [4.5, 4.7, 4.8]
                      .map(
                        (rating) => ChoiceChip(
                          label: Text('${rating.toStringAsFixed(1)}+'),
                          selected: tempRating == rating,
                          onSelected: (_) {
                            setModalState(() {
                              tempRating =
                                  tempRating == rating ? null : rating;
                            });
                          },
                        ),
                      )
                      .toList(),
                ),
                const SizedBox(height: 16),
                Text(
                  'Özellikler',
                  style: Theme.of(context).textTheme.titleMedium,
                ),
                const SizedBox(height: 10),
                Wrap(
                  spacing: 8,
                  runSpacing: 8,
                  children: features
                      .map(
                        (feature) => FilterChip(
                          label: Text(feature),
                          selected: tempFeatures.contains(feature),
                          onSelected: (selected) {
                            setModalState(() {
                              if (selected) {
                                tempFeatures.add(feature);
                              } else {
                                tempFeatures.remove(feature);
                              }
```

```
                                  });
                              },
                            ),
                          )
                          .toList(),
                    ),
                    const SizedBox(height: 20),
                    SizedBox(
                      width: double.infinity,
                      child: FilledButton(
                        onPressed: () {
                          Navigator.of(context).pop(
                            _FilterSelection(
                              area: tempArea,
                              minimumRating: tempRating,
                              features: tempFeatures,
                            ),
                          );
                        },
                        child: const Text('Uygula'),
                      ),
                    ),
                  ],
                ),
              ),
            ),
          ),
        );
      },
    );

    if (result == null) {
      return;
    }

    setState(() {
      _selectedArea = result.area;
      _minimumRating = result.minimumRating;
      _selectedFeatures
        ..clear()
        ..addAll(result.features);
    });
  }

  @override
  Widget build(BuildContext context) {
    final width = MediaQuery.sizeOf(context).width;
    final padding = width >= 1080 ? 24.0 : 16.0;

    return Consumer<HotelAppState>(
      builder: (context, state, _) {
        final hotels = _visibleHotels(state);

        return Align(
          alignment: Alignment.topCenter,
          child: ConstrainedBox(
            constraints: const BoxConstraints(maxWidth: 1240),
            child: SingleChildScrollView(
              padding: EdgeInsets.fromLTRB(padding, 16, padding, 28),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Row(
                    children: [
```

```
                    Expanded(
                      child: Text(
                        'Oteller',
                        style: Theme.of(context).textTheme.displayMedium,
                      ),
                    ),
                    _FilterCountPill(
                      label: '${hotels.length} ilan',
                      icon: Icons.grid_view_rounded,
                    ),
                  ],
                ),
                const SizedBox(height: 10),
                SoftPanel(
                  padding: const EdgeInsets.all(14),
                  radius: 18,
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Row(
                        children: [
                          Text(
                            'Arama',
                            style: Theme.of(context).textTheme.titleMedium,
                          ),
                          const Spacer(),
                          OutlinedButton.icon(
                            onPressed: () => _openFilterSheet(state),
                            icon: const Icon(
                              Icons.tune_rounded,
                              size: 18,
                            ),
                            label: Text(
                              _activeFilterCount == 0
                                  ? 'Filtreler'
                                  : 'Filtreler ($_activeFilterCount)',
                            ),
                          ),
                        ],
                      ),
                      const SizedBox(height: 12),
                      TextField(
                        controller: _searchController,
                        onChanged: (_) => setState(() {}),
                        decoration: InputDecoration(
                          hintText: 'Otel veya bölge ara',
                          prefixIcon:
                              const Icon(Icons.search_rounded, size: 20),
                          suffixIcon: _searchController.text.isEmpty
                              ? null
                              : IconButton(
                                  onPressed: () {
                                    _searchController.clear();
                                    setState(() {});
                                  },
                                  icon: const Icon(
                                    Icons.close_rounded,
                                    size: 18,
                                  ),
                                ),
                        ),
                      ),
                      const SizedBox(height: 12),
                      Row(
                        children: [
                          Expanded(
```

```
                                child: Text(
                                  '${hotels.length} ilan',
                                  style: Theme.of(context).textTheme.bodyMedium,
                                ),
                              ),
                            ],
                          ),
                        ],
                      ),
                    ),
                    const SizedBox(height: 12),
                    if (hotels.isEmpty)
                      const SoftPanel(
                        child: Text('ıAramanza uygun otel ıbulunamad.'),
                      )
                    else
                      LayoutBuilder(
                        builder: (context, constraints) {
                          const gap = 12.0;
                          final columns = constraints.maxWidth >= 360 ? 2 : 1;
                          final totalGap = gap * (columns - 1);
                          final cardWidth =
                              (constraints.maxWidth - totalGap) / columns;
                          final cardHeight =
                              constraints.maxWidth >= 920 ? 360.0 : 336.0;

                          return Wrap(
                            spacing: gap,
                            runSpacing: gap,
                            children: hotels
                                .map(
                                  (hotel) => SizedBox(
                                    width: cardWidth,
                                    height: cardHeight,
                                    child: _HotelCard(
                                      hotel: hotel,
                                      selected: state.selectedHotelId == hotel.id,
                                      onTap: () {
                                        state.selectHotel(hotel.id);
                                        widget.onNavigate(1);
                                      },
                                    ),
                                  ),
                                )
                                .toList(),
                          );
                        },
                      ),
                  ],
                ),
              ),
            ),
          ),
        );
      },
    );
  }
}

class _HotelCard extends StatelessWidget {
  const _HotelCard({
    required this.hotel,
    required this.selected,
    required this.onTap,
  });

  final HotelListing hotel;
```

```
  final bool selected;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);

    return InkWell(
      borderRadius: BorderRadius.circular(20),
      onTap: onTap,
      child: SoftPanel(
        backgroundColor:
            selected ? hotel.accent.withValues(alpha: 0.07) : Colors.white,
        borderColor: selected ? hotel.accent.withValues(alpha: 0.22) : null,
        padding: const EdgeInsets.all(8),
        radius: 20,
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Expanded(
              child: ClipRRect(
                borderRadius: BorderRadius.circular(16),
                child: Stack(
                  fit: StackFit.expand,
                  children: [
                    Image.asset(hotel.imagePath, fit: BoxFit.cover),
                    Positioned(
                      left: 8,
                      right: 8,
                      bottom: 8,
                      child: Row(
                        children: [
                          Expanded(
                            child: Container(
                              padding: const EdgeInsets.symmetric(
                                horizontal: 8,
                                vertical: 6,
                              ),
                              decoration: BoxDecoration(
                                color: Colors.black.withValues(alpha: 0.52),
                                borderRadius: BorderRadius.circular(12),
                              ),
                              child: Text(
                                '${hotel.area}, ${hotel.city}',
                                maxLines: 1,
                                overflow: TextOverflow.ellipsis,
                                style: theme.textTheme.bodySmall?.copyWith(
                                  color: Colors.white,
                                  fontWeight: FontWeight.w600,
                                ),
                              ),
                            ),
                          ),
                        ),
                        if (selected) ...[
                          const SizedBox(width: 8),
                          Container(
                            width: 32,
                            height: 32,
                            decoration: BoxDecoration(
                              color: Colors.white,
                              borderRadius: BorderRadius.circular(999),
                            ),
                            child: Icon(
                              Icons.check_circle_rounded,
                              color: hotel.accent,
                              size: 18,
```

```
                              ),
                            ),
                          ],
                        ],
                      ),
                    ),
                  ],
                ),
              ),
            ),
            const SizedBox(height: 10),
            Text(
              hotel.name,
              maxLines: 2,
              overflow: TextOverflow.ellipsis,
              style: theme.textTheme.titleMedium,
            ),
            const SizedBox(height: 4),
            Text(
              hotel.tags.take(2).join(' • '),
              maxLines: 1,
              overflow: TextOverflow.ellipsis,
              style: theme.textTheme.bodyMedium,
            ),
            const SizedBox(height: 8),
            Row(
              children: [
                Icon(
                  Icons.star_rounded,
                  size: 16,
                  color: theme.colorScheme.secondary,
                ),
                const SizedBox(width: 4),
                Text(
                  hotel.rating.toStringAsFixed(1),
                  style: theme.textTheme.bodyMedium,
                ),
                const SizedBox(width: 6),
                Expanded(
                  child: Text(
                    '${hotel.reviewCount}+ yorum',
                    maxLines: 1,
                    overflow: TextOverflow.ellipsis,
                    style: theme.textTheme.bodyMedium,
                  ),
                ),
              ],
            ),
            const SizedBox(height: 10),
            SizedBox(
              width: double.infinity,
              child: FilledButton.tonal(
                onPressed: onTap,
                child: const Text('Detay'),
              ),
            ),
          ],
        ),
      ),
    );
  }
}

class _FilterCountPill extends StatelessWidget {
  const _FilterCountPill({
    required this.label,
```

```
    required this.icon,
  });

  final String label;
  final IconData icon;

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 8),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(999),
        border: Border.all(
          color: theme.colorScheme.primary.withValues(alpha: 0.08),
        ),
      ),
      child: Row(
        mainAxisSize: MainAxisSize.min,
        children: [
          Icon(icon, size: 16, color: theme.colorScheme.secondary),
          const SizedBox(width: 8),
          Text(label, style: theme.textTheme.labelLarge),
        ],
      ),
    );
  }
}

class _FilterSelection {
  const _FilterSelection({
    required this.area,
    required this.minimumRating,
    required this.features,
  });

  final String? area;
  final double? minimumRating;
  final Set<String> features;
}
```

Açıklama: Listeleme ekranı projedeki ilk kullanıcı temasını temsil eder. Arama sorgusunun doğrudan veri kümesine uygulanması ve filtrelerin modal alt sayfa üzerinden verilmesi, mobil arayüzlerde sık kullanılan pratik bir yaklaşımı göstermektedir.

10. lib/screens/booking_screen.dart - Rezervasyon formu, tarih seçimi ve fiyat özeti

Dosyanın görevi: Projenin en yoğun ekranı burasıdır. Oda varyantı seçimi, tarih aralığı belirleme, misafir sayısı değiştirme, ek hizmet seçme, form doğrulama ve toplam ücret hesabı aynı akış içinde yürütülür.

Satır sayısı: 1231

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/hotel_models.dart';
import '../state/hotel_app_state.dart';
import '../utils/formatters.dart';
import '../widgets/hotel_widgets.dart';

class BookingScreen extends StatefulWidget {
  const BookingScreen({super.key});

  @override
  State<BookingScreen> createState() => _BookingScreenState();
}

class _BookingScreenState extends State<BookingScreen> {
  final _formKey = GlobalKey<FormState>();
  final _nameController = TextEditingController();
  final _emailController = TextEditingController();
  final _noteController = TextEditingController();

  DateTime? _checkInDate;
  DateTime? _checkOutDate;
  int _guestCount = 2;
  final Set<String> _selectedExtraIds = {};

  static const List<_BookingExtra> _extras = [
    _BookingExtra(
      id: 'breakfast',
      label: 'ıKahvalt',
      caption: 'şKii şıba / gece',
      price: 450,
      perNight: true,
      icon: Icons.restaurant_rounded,
    ),
    _BookingExtra(
      id: 'transfer',
      label: 'Transfer',
      caption: 'Tek yön',
      price: 900,
      perNight: false,
      icon: Icons.airport_shuttle_rounded,
    ),
    _BookingExtra(
      id: 'late_checkout',
      label: 'Geç çıışk',
      caption: '14:00 çıışk',
      price: 600,
      perNight: false,
      icon: Icons.schedule_rounded,
    ),
  ];

  @override
  void dispose() {
    _nameController.dispose();
    _emailController.dispose();
    _noteController.dispose();
    super.dispose();
  }
```

```
61
62  Future<void> _pickDateRange(BuildContext context) async {
63    final now = DateUtils.dateOnly(DateTime.now());
64    final initialStart = _checkInDate != null && !_checkInDate!.isBefore(now)
65        ? _checkInDate!
66        : now.add(const Duration(days: 7));
67    final initialEnd =
68        _checkOutDate != null && _checkOutDate!.isAfter(initialStart)
69            ? _checkOutDate!
70            : initialStart.add(const Duration(days: 2));
71
72    final range = await showDateRangePicker(
73      context: context,
74      helpText: 'Tarih ığıaral',
75      cancelText: İ'ptal',
76      confirmText: 'Uygula',
77      saveText: 'Uygula',
78      initialDateRange: DateTimeRange(
79        start: initialStart,
80        end: initialEnd,
81      ),
82      firstDate: now,
83      lastDate: now.add(const Duration(days: 365)),
84    );
85
86    if (range == null) {
87      return;
88    }
89
90    setState(() {
91      _checkInDate = range.start;
92      _checkOutDate = range.end;
93    });
94  }
95
96  Future<void> _submit(HotelAppState state) async {
97    final room = state.selectedRoom;
98    if (room == null) {
99      return;
100   }
101
102   final boundedGuests =
103       _guestCount > room.maxGuests ? room.maxGuests : _guestCount;
104   final formIsValid = _formKey.currentState!.validate();
105   final hasValidDates = _checkInDate != null &&
106       _checkOutDate != null &&
107       _checkOutDate!.isAfter(_checkInDate!);
108
109   if (!formIsValid || !hasValidDates) {
110     final message = !formIsValid
111         ? 'Eksik ıalanlar doldurun.'
112         : 'Çıış tarihi şgiri tarihinden sonra ıolmal.';
113     ScaffoldMessenger.of(context)
114         .showSnackBar(SnackBar(content: Text(message)));
115     return;
116   }
117
118   final extras = _selectedExtras;
119   await state.saveReservation(
120     guestName: _nameController.text.trim(),
121     email: _emailController.text.trim(),
122     checkIn: _checkInDate!,
123     checkOut: _checkOutDate!,
124     guests: boundedGuests,
125     specialRequest: _noteController.text.trim(),
126     extras: extras.map((item) => item.label).toList(),
```

```
      extrasTotal: _extrasTotal(boundedGuests, _nightCount),
    );

    if (!mounted) {
      return;
    }

    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Rezervasyon şoluturuldu.')),
    );

    setState(() {
      _nameController.clear();
      _emailController.clear();
      _noteController.clear();
      _checkInDate = null;
      _checkOutDate = null;
      _guestCount = 2;
      _selectedExtraIds.clear();
    });
  }

  int get _nightCount {
    if (_checkInDate == null || _checkOutDate == null) {
      return 1;
    }

    final nights = _checkOutDate!.difference(_checkInDate!).inDays;
    return nights > 0 ? nights : 1;
  }

  List<_BookingExtra> get _selectedExtras => _extras
      .where((item) => _selectedExtraIds.contains(item.id))
      .toList(growable: false);

  int _extrasTotal(int guests, int nights) {
    return _selectedExtras.fold(
      0,
      (sum, item) => sum + _extraCost(item, guests, nights),
    );
  }

  int _extraCost(_BookingExtra item, int guests, int nights) {
    final multiplier = item.perNight ? nights : 1;
    final guestMultiplier = item.id == 'breakfast' ? guests : 1;
    return item.price * multiplier * guestMultiplier;
  }

  @override
  Widget build(BuildContext context) {
    final width = MediaQuery.sizeOf(context).width;
    final padding = width >= 1080 ? 24.0 : 16.0;

    return Consumer<HotelAppState>(
      builder: (context, state, _) {
        final hotel = state.selectedHotel;
        final room = state.selectedRoom;

        if (hotel == null || room == null) {
          return Align(
            alignment: Alignment.topCenter,
            child: ConstrainedBox(
              constraints: const BoxConstraints(maxWidth: 1240),
              child: _EmptyBookingState(padding: padding),
            ),
          );
```

```
193         }
194
195         final rooms = state.roomsForSelectedHotel;
196         final guestCount =
197             _guestCount > room.maxGuests ? room.maxGuests : _guestCount;
198         final extrasTotal = _extrasTotal(guestCount, _nightCount);
199         final roomTotal = room.nightlyRate * _nightCount;
200         final grandTotal = roomTotal + extrasTotal;
201
202         return Align(
203           alignment: Alignment.topCenter,
204           child: ConstrainedBox(
205             constraints: const BoxConstraints(maxWidth: 1240),
206             child: SingleChildScrollView(
207               padding: EdgeInsets.fromLTRB(padding, 18, padding, 28),
208               child: Column(
209                 crossAxisAlignment: CrossAxisAlignment.start,
210                 children: [
211                   PhotoHero(
212                     imagePath: hotel.imagePath,
213                     eyebrow: '${hotel.city} • ${hotel.area}',
214                     title: hotel.name,
215                     subtitle:
216                         '${hotel.rating.toStringAsFixed(1)} puan • ${hotel.reviewCount}+
    yorum',
217                     height: 180,
218                   ),
219                   const SizedBox(height: 12),
220                   SectionHeader(
221                     eyebrow: 'Varyantlar',
222                     title: 'Odalar',
223                     subtitle: '${rooms.length} seçenek',
224                   ),
225                   const SizedBox(height: 12),
226                   LayoutBuilder(
227                     builder: (context, constraints) {
228                       const gap = 10.0;
229                       final columns = constraints.maxWidth >= 300 ? 3 : 2;
230                       final totalGap = gap * (columns - 1);
231                       final cardWidth =
232                           (constraints.maxWidth - totalGap) / columns;
233                       final cardHeight =
234                           constraints.maxWidth >= 920 ? 180.0 : 172.0;
235
236                       return Wrap(
237                         spacing: gap,
238                         runSpacing: gap,
239                         children: rooms
240                             .map(
241                               (item) => SizedBox(
242                                 width: cardWidth,
243                                 height: cardHeight,
244                                 child: _BookingRoomCard(
245                                   room: item,
246                                   selected: item.id == room.id,
247                                   onTap: () {
248                                     state.selectRoom(item.id);
249                                     if (_guestCount > item.maxGuests) {
250                                       setState(() {
251                                         _guestCount = item.maxGuests;
252                                       });
253                                     }
254                                   },
255                                 ),
256                               ),
257                             )
```

```
                    .toList(),
              );
            },
          ),
          const SizedBox(height: 16),
          LayoutBuilder(
            builder: (context, constraints) {
              final isWide = constraints.maxWidth >= 980;
              final form = SoftPanel(
                padding: const EdgeInsets.all(16),
                child: Form(
                  key: _formKey,
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Row(
                        children: [
                          Expanded(
                            child: Text(
                              'Rezervasyon',
                              style: Theme.of(context)
                                  .textTheme
                                  .titleLarge,
                            ),
                          ),
                          Container(
                            padding: const EdgeInsets.symmetric(
                              horizontal: 10,
                              vertical: 6,
                            ),
                            decoration: BoxDecoration(
                              color: Theme.of(context)
                                  .colorScheme
                                  .secondary
                                  .withValues(alpha: 0.1),
                              borderRadius: BorderRadius.circular(999),
                            ),
                            child: Text(
                              formatCurrency(grandTotal),
                              style: Theme.of(context)
                                  .textTheme
                                  .labelLarge
                                  ?.copyWith(
                                    color: Theme.of(context)
                                        .colorScheme
                                        .secondary,
                                  ),
                            ),
                          ),
                        ],
                      ),
                      const SizedBox(height: 14),
                      LayoutBuilder(
                        builder: (context, localConstraints) {
                          final gap = 8.0;
                          final statWidth = localConstraints.maxWidth >=
                                  560
                              ? (localConstraints.maxWidth -
                                      (gap * 2)) /
                                  3
                              : (localConstraints.maxWidth - gap) / 2;

                          return Wrap(
                            spacing: gap,
                            runSpacing: gap,
                            children: [
```

```
                SizedBox(
                  width: statWidth,
                  child: _BookingMetricCard(
                    label: 'Oda',
                    value: room.name,
                    icon: Icons.bed_rounded,
                  ),
                ),
                SizedBox(
                  width: statWidth,
                  child: _BookingMetricCard(
                    label: 'Gece',
                    value: '$_nightCount',
                    icon: Icons.nights_stay_rounded,
                  ),
                ),
                SizedBox(
                  width: statWidth,
                  child: _BookingMetricCard(
                    label: 'Misafir',
                    value: '$guestCount',
                    icon: Icons.group_rounded,
                  ),
                ),
              ],
            );
          },
        ),
        const SizedBox(height: 16),
        const _BookingSectionLabel('Misafir bilgileri'),
        const SizedBox(height: 10),
        LayoutBuilder(
          builder: (context, localConstraints) {
            final wideFields =
                localConstraints.maxWidth >= 620;

            final nameField = TextFormField(
              controller: _nameController,
              decoration: const InputDecoration(
                labelText: 'Ad soyad',
              ),
              validator: (value) {
                if (value == null ||
                    value.trim().isEmpty) {
                  return 'Zorunlu alan';
                }
                return null;
              },
            );

            final emailField = TextFormField(
              controller: _emailController,
              decoration: const InputDecoration(
                labelText: 'E-posta',
              ),
              validator: (value) {
                final email = value?.trim() ?? '';
                if (email.isEmpty ||
                    !email.contains('@')) {
                  return 'Geçerli e-posta girin';
                }
                return null;
              },
            );

            if (!wideFields) {
```

```
            return Column(
              children: [
                nameField,
                const SizedBox(height: 12),
                emailField,
              ],
            );
          }

          return Row(
            children: [
              Expanded(child: nameField),
              const SizedBox(width: 12),
              Expanded(child: emailField),
            ],
          );
        },
      ),
      const SizedBox(height: 14),
      const _BookingSectionLabel('Tarih ve konaklama'),
      const SizedBox(height: 10),
      _DateRangeCard(
        start: _checkInDate,
        end: _checkOutDate,
        nightCount: _nightCount,
        onTap: () => _pickDateRange(context),
      ),
      const SizedBox(height: 14),
      Container(
        padding: const EdgeInsets.symmetric(
          horizontal: 12,
          vertical: 10,
        ),
        decoration: BoxDecoration(
          color: Theme.of(context)
              .colorScheme
              .primary
              .withValues(alpha: 0.04),
          borderRadius: BorderRadius.circular(14),
          border: Border.all(
            color: Theme.of(context)
                .colorScheme
                .primary
                .withValues(alpha: 0.08),
          ),
        ),
        child: Row(
          children: [
            Text(
              'Misafir',
              style: Theme.of(context)
                  .textTheme
                  .titleMedium,
            ),
            const Spacer(),
            IconButton.filledTonal(
              onPressed: guestCount > 1
                  ? () {
                      setState(() {
                        _guestCount = guestCount - 1;
                      });
                    }
                  : null,
              icon: const Icon(Icons.remove_rounded),
            ),
            SizedBox(
```

```
                      width: 42,
                      child: Text(
                        '$guestCount',
                        textAlign: TextAlign.center,
                        style: Theme.of(context)
                            .textTheme
                            .titleLarge,
                      ),
                    ),
                    IconButton.filledTonal(
                      onPressed: guestCount < room.maxGuests
                          ? () {
                              setState(() {
                                _guestCount = guestCount + 1;
                              });
                            }
                          : null,
                      icon: const Icon(Icons.add_rounded),
                    ),
                  ],
                ),
              ),
              const SizedBox(height: 14),
              const _BookingSectionLabel('Ek hizmetler'),
              const SizedBox(height: 10),
              LayoutBuilder(
                builder: (context, localConstraints) {
                  const gap = 8.0;
                  final columns =
                      localConstraints.maxWidth >= 720 ? 3 : 1;
                  final itemWidth = columns == 1
                      ? localConstraints.maxWidth
                      : (localConstraints.maxWidth -
                              ((columns - 1) * gap)) /
                          columns;

                  return Wrap(
                    spacing: gap,
                    runSpacing: gap,
                    children: _extras
                        .map(
                          (item) => SizedBox(
                            width: itemWidth,
                            child: _ExtraOptionCard(
                              item: item,
                              selected: _selectedExtraIds
                                  .contains(item.id),
                              totalCost: _extraCost(
                                item,
                                guestCount,
                                _nightCount,
                              ),
                              onTap: () {
                                setState(() {
                                  if (_selectedExtraIds
                                      .contains(item.id)) {
                                    _selectedExtraIds
                                        .remove(item.id);
                                  } else {
                                    _selectedExtraIds
                                        .add(item.id);
                                  }
                                });
                              },
                            ),
                          ),
```

```
                                )
                                .toList(),
                          );
                      },
                    ),
                    const SizedBox(height: 12),
                    const _BookingSectionLabel('Not'),
                    const SizedBox(height: 10),
                    TextFormField(
                      controller: _noteController,
                      minLines: 3,
                      maxLines: 4,
                      decoration: const InputDecoration(
                        labelText: 'Not',
                      ),
                    ),
                    const SizedBox(height: 18),
                    SizedBox(
                      width: double.infinity,
                      child: FilledButton(
                        onPressed: () => _submit(state),
                        child: const Text('Rezervasyonu tamamla'),
                      ),
                    ),
                  ],
                ),
              ),
            );

            final summary = SoftPanel(
              padding: const EdgeInsets.all(16),
              backgroundColor: Theme.of(context)
                  .colorScheme
                  .primary
                  .withValues(alpha: 0.05),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  _BookingSummaryHero(
                    hotelName: hotel.name,
                    roomName: room.name,
                    imagePath: room.imagePath,
                    dateLabel:
                        _checkInDate != null && _checkOutDate != null
                            ? formatDateRange(
                                _checkInDate!,
                                _checkOutDate!,
                              )
                            : 'Tarih seçilmedi',
                  ),
                  const SizedBox(height: 12),
                  Container(
                    width: double.infinity,
                    padding: const EdgeInsets.all(14),
                    decoration: BoxDecoration(
                      color: Colors.white,
                      borderRadius: BorderRadius.circular(16),
                      border: Border.all(
                        color: Theme.of(context)
                            .colorScheme
                            .primary
                            .withValues(alpha: 0.08),
                      ),
                    ),
                    child: Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
```

```
                    children: [
                      Text(
                        'Genel toplam',
                        style:
                            Theme.of(context).textTheme.bodyMedium,
                      ),
                      const SizedBox(height: 6),
                      Text(
                        formatCurrency(grandTotal),
                        style: Theme.of(context)
                            .textTheme
                            .headlineMedium,
                      ),
                    ],
                  ),
                ),
                const SizedBox(height: 12),
                _SummaryInfoTile(
                  icon: Icons.calendar_today_rounded,
                  label: 'Tarih',
                  value:
                      _checkInDate != null && _checkOutDate != null
                          ? formatDateRange(
                              _checkInDate!,
                              _checkOutDate!,
                            )
                          : '-',
                ),
                const SizedBox(height: 8),
                _SummaryInfoTile(
                  icon: Icons.group_rounded,
                  label: 'Misafir',
                  value: '$guestCount',
                ),
                const SizedBox(height: 8),
                _SummaryInfoTile(
                  icon: Icons.nights_stay_rounded,
                  label: 'Gece',
                  value: '$_nightCount',
                ),
                const SizedBox(height: 14),
                _SummaryLine(
                  label: 'Oda',
                  value: '${formatCurrency(room.nightlyRate)}/gece',
                ),
                _SummaryLine(
                  label: 'Oda ıtoplam',
                  value: formatCurrency(roomTotal),
                ),
                _SummaryLine(
                  label: 'Ek hizmet',
                  value: formatCurrency(extrasTotal),
                ),
                _SummaryLine(
                  label: 'Genel toplam',
                  value: formatCurrency(grandTotal),
                ),
                if (_selectedExtras.isNotEmpty) ...[
                  const SizedBox(height: 12),
                  const _BookingSectionLabel('Ek hizmetler'),
                  const SizedBox(height: 8),
                  Wrap(
                    spacing: 6,
                    runSpacing: 6,
                    children: _selectedExtras
                        .map(
```

```
                                    (item) => FeaturePill(
                                      label:
                                          '${item.label} ${formatCurrency(_extraCost(
    item, guestCount, _nightCount))}',
                                    ),
                                  )
                                  .toList(),
                            ),
                          ],
                          const SizedBox(height: 12),
                          const _BookingSectionLabel('Oda özellikleri'),
                          const SizedBox(height: 8),
                          Wrap(
                            spacing: 6,
                            runSpacing: 6,
                            children: room.perks
                                .take(2)
                                .map((perk) => FeaturePill(label: perk))
                                .toList(),
                          ),
                        ],
                      ),
                    );

                    if (isWide) {
                      return Row(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: [
                          Expanded(flex: 5, child: form),
                          const SizedBox(width: 16),
                          Expanded(flex: 4, child: summary),
                        ],
                      );
                    }

                    return Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: [
                        form,
                        const SizedBox(height: 16),
                        summary,
                      ],
                    );
                  },
                ),
              ],
            ),
          ),
        ),
      );
    },
  );
  }
}

class _EmptyBookingState extends StatelessWidget {
  const _EmptyBookingState({
    required this.padding,
  });

  final double padding;

  @override
  Widget build(BuildContext context) {
    return SingleChildScrollView(
      padding: EdgeInsets.fromLTRB(padding, 24, padding, 28),
```

```
      child: Center(
        child: ConstrainedBox(
          constraints: const BoxConstraints(maxWidth: 560),
          child: SoftPanel(
            padding: const EdgeInsets.all(24),
            child: Column(
              children: [
                Icon(
                  Icons.hotel_class_rounded,
                  size: 44,
                  color: Theme.of(context).colorScheme.secondary,
                ),
                const SizedBox(height: 14),
                Text(
                  'Rezervasyon',
                  style: Theme.of(context).textTheme.headlineMedium,
                ),
                const SizedBox(height: 8),
                Text(
                  'Seçili tesis yok',
                  textAlign: TextAlign.center,
                  style: Theme.of(context).textTheme.bodyLarge,
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}

class _BookingRoomCard extends StatelessWidget {
  const _BookingRoomCard({
    required this.room,
    required this.selected,
    required this.onTap,
  });

  final RoomOffer room;
  final bool selected;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    return InkWell(
      borderRadius: BorderRadius.circular(18),
      onTap: onTap,
      child: SoftPanel(
        backgroundColor:
            selected ? room.accent.withValues(alpha: 0.07) : Colors.white,
        borderColor: selected ? room.accent.withValues(alpha: 0.24) : null,
        padding: const EdgeInsets.all(8),
        radius: 18,
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Expanded(
              child: ClipRRect(
                borderRadius: BorderRadius.circular(14),
                child: SizedBox(
                  width: double.infinity,
                  child: Image.asset(room.imagePath, fit: BoxFit.cover),
                ),
              ),
```

```
            ),
            const SizedBox(height: 6),
            Text(
              room.name,
              maxLines: 1,
              overflow: TextOverflow.ellipsis,
              style: theme.textTheme.labelLarge,
            ),
            const SizedBox(height: 2),
            Text(
              room.sizeLabel,
              maxLines: 1,
              overflow: TextOverflow.ellipsis,
              style: theme.textTheme.bodySmall,
            ),
            const SizedBox(height: 2),
            Text(
              '${room.maxGuests} misafir',
              maxLines: 1,
              overflow: TextOverflow.ellipsis,
              style: theme.textTheme.bodySmall?.copyWith(
                color: theme.colorScheme.secondary,
              ),
            ),
            const SizedBox(height: 6),
            Row(
              children: [
                Expanded(
                  child: Text(
                    formatCurrency(room.nightlyRate),
                    maxLines: 1,
                    overflow: TextOverflow.ellipsis,
                    style: theme.textTheme.labelLarge,
                  ),
                ),
                FilledButton.tonal(
                  onPressed: onTap,
                  style: FilledButton.styleFrom(
                    padding: const EdgeInsets.symmetric(
                      horizontal: 10,
                      vertical: 8,
                    ),
                    tapTargetSize: MaterialTapTargetSize.shrinkWrap,
                    minimumSize: Size.zero,
                  ),
                  child: Text(selected ? 'Seçili' : 'Seç'),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}

class _BookingMetricCard extends StatelessWidget {
  const _BookingMetricCard({
    required this.label,
    required this.value,
    required this.icon,
  });

  final String label;
  final String value;
  final IconData icon;
```

```

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 10),
      decoration: BoxDecoration(
        color: theme.colorScheme.primary.withValues(alpha: 0.04),
        borderRadius: BorderRadius.circular(14),
        border: Border.all(
          color: theme.colorScheme.primary.withValues(alpha: 0.08),
        ),
      ),
      child: Row(
        children: [
          Icon(icon, size: 18, color: theme.colorScheme.secondary),
          const SizedBox(width: 10),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(label, style: theme.textTheme.bodySmall),
                const SizedBox(height: 2),
                Text(
                  value,
                  maxLines: 1,
                  overflow: TextOverflow.ellipsis,
                  style: theme.textTheme.labelLarge,
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}

class _DateRangeCard extends StatelessWidget {
  const _DateRangeCard({
    required this.start,
    required this.end,
    required this.nightCount,
    required this.onTap,
  });

  final DateTime? start;
  final DateTime? end;
  final int nightCount;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    final hasRange = start != null && end != null;

    return InkWell(
      borderRadius: BorderRadius.circular(16),
      onTap: onTap,
      child: Container(
        width: double.infinity,
        padding: const EdgeInsets.all(14),
        decoration: BoxDecoration(
          color: theme.colorScheme.primary.withValues(alpha: 0.04),
          borderRadius: BorderRadius.circular(16),
          border: Border.all(
```

```
            color: theme.colorScheme.primary.withValues(alpha: 0.08),
          ),
        ),
        child: Row(
          children: [
            Container(
              width: 40,
              height: 40,
              decoration: BoxDecoration(
                color: Colors.white,
                borderRadius: BorderRadius.circular(12),
              ),
              child: Icon(
                Icons.calendar_month_rounded,
                color: theme.colorScheme.secondary,
                size: 20,
              ),
            ),
            const SizedBox(width: 12),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    'Konaklama tarihi',
                    style: theme.textTheme.bodySmall,
                  ),
                  const SizedBox(height: 4),
                  Text(
                    hasRange
                        ? formatDateRange(start!, end!)
                        : 'Tarih ığıaral seçin',
                    style: theme.textTheme.labelLarge,
                  ),
                ],
              ),
            ),
            if (hasRange) ...[
              const SizedBox(width: 10),
              Container(
                padding: const EdgeInsets.symmetric(
                  horizontal: 10,
                  vertical: 7,
                ),
                decoration: BoxDecoration(
                  color: Colors.white,
                  borderRadius: BorderRadius.circular(999),
                ),
                child: Text(
                  '$nightCount gece',
                  style: theme.textTheme.labelLarge,
                ),
              ),
            ],
          ],
        ),
      ),
    );
  }
}

class _ExtraOptionCard extends StatelessWidget {
  const _ExtraOptionCard({
    required this.item,
    required this.selected,
    required this.totalCost,
```

```
    required this.onTap,
  });

  final _BookingExtra item;
  final bool selected;
  final int totalCost;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);

    return InkWell(
      borderRadius: BorderRadius.circular(16),
      onTap: onTap,
      child: Container(
        padding: const EdgeInsets.all(14),
        decoration: BoxDecoration(
          color: selected
              ? theme.colorScheme.secondary.withValues(alpha: 0.08)
              : Colors.white,
          borderRadius: BorderRadius.circular(16),
          border: Border.all(
            color: selected
                ? theme.colorScheme.secondary.withValues(alpha: 0.22)
                : theme.colorScheme.primary.withValues(alpha: 0.08),
          ),
        ),
        child: Row(
          children: [
            Container(
              width: 42,
              height: 42,
              decoration: BoxDecoration(
                color: Colors.white,
                borderRadius: BorderRadius.circular(14),
              ),
              child: Icon(
                item.icon,
                size: 20,
                color: theme.colorScheme.secondary,
              ),
            ),
            const SizedBox(width: 12),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(item.label, style: theme.textTheme.titleMedium),
                  const SizedBox(height: 2),
                  Text(item.caption, style: theme.textTheme.bodySmall),
                  const SizedBox(height: 6),
                  Text(
                    formatCurrency(totalCost),
                    style: theme.textTheme.labelLarge,
                  ),
                ],
              ),
            ),
            const SizedBox(width: 10),
            Icon(
              selected
                  ? Icons.check_circle_rounded
                  : Icons.radio_button_unchecked_rounded,
              color: selected
                  ? theme.colorScheme.secondary
```

```
                  : theme.colorScheme.primary.withValues(alpha: 0.36),
              size: 20,
            ),
          ],
        ),
      ),
    );
  }
}

class _BookingSectionLabel extends StatelessWidget {
  const _BookingSectionLabel(this.label);

  final String label;

  @override
  Widget build(BuildContext context) {
    return Text(
      label,
      style: Theme.of(context).textTheme.titleMedium,
    );
  }
}

class _BookingSummaryHero extends StatelessWidget {
  const _BookingSummaryHero({
    required this.hotelName,
    required this.roomName,
    required this.imagePath,
    required this.dateLabel,
  });

  final String hotelName;
  final String roomName;
  final String imagePath;
  final String dateLabel;

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    return Container(
      padding: const EdgeInsets.all(12),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(16),
        border: Border.all(
          color: theme.colorScheme.primary.withValues(alpha: 0.08),
        ),
      ),
      child: Row(
        children: [
          ClipRRect(
            borderRadius: BorderRadius.circular(14),
            child: SizedBox(
              width: 72,
              height: 72,
              child: Image.asset(imagePath, fit: BoxFit.cover),
            ),
          ),
          const SizedBox(width: 12),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  hotelName,
```

```
                  maxLines: 1,
                  overflow: TextOverflow.ellipsis,
                  style: theme.textTheme.titleMedium,
                ),
                const SizedBox(height: 4),
                Text(
                  roomName,
                  maxLines: 1,
                  overflow: TextOverflow.ellipsis,
                  style: theme.textTheme.bodyMedium,
                ),
                const SizedBox(height: 6),
                Text(
                  dateLabel,
                  maxLines: 2,
                  overflow: TextOverflow.ellipsis,
                  style: theme.textTheme.bodySmall?.copyWith(
                    color: theme.colorScheme.secondary,
                  ),
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}

class _SummaryInfoTile extends StatelessWidget {
  const _SummaryInfoTile({
    required this.icon,
    required this.label,
    required this.value,
  });

  final IconData icon;
  final String label;
  final String value;

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 10),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(14),
      ),
      child: Row(
        children: [
          Icon(icon, size: 18, color: theme.colorScheme.secondary),
          const SizedBox(width: 10),
          Expanded(
            child: Text(label, style: theme.textTheme.bodyMedium),
          ),
          const SizedBox(width: 10),
          Flexible(
            child: Text(
              value,
              textAlign: TextAlign.right,
              style: theme.textTheme.labelLarge,
            ),
          ),
        ],
      ),
```

```
    );
  }
}

class _SummaryLine extends StatelessWidget {
  const _SummaryLine({
    required this.label,
    required this.value,
  });

  final String label;
  final String value;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 6),
      child: Row(
        children: [
          Expanded(child: Text(label)),
          const SizedBox(width: 12),
          Flexible(
            child: Text(
              value,
              textAlign: TextAlign.right,
              style: Theme.of(context).textTheme.titleMedium,
            ),
          ),
        ],
      ),
    );
  }
}

class _BookingExtra {
  const _BookingExtra({
    required this.id,
    required this.label,
    required this.caption,
    required this.price,
    required this.perNight,
    required this.icon,
  });

  final String id;
  final String label;
  final String caption;
  final int price;
  final bool perNight;
  final IconData icon;
}
```

Açıklama: Bu dosya ders açısından en güçlü bölümdür; çünkü hem kullanıcı arayüzünü hem de iş kuralı mantığını birlikte taşır. ‘showDateRangePicker‘, form validasyonu, ‘LayoutBuilder‘ ile duyarlı yerleşim ve ek hizmet hesaplamaları aynı ekranda bir araya getirilmiştir.

11. lib/screens/reservations_screen.dart - Yaklaşan ve geçmiş rezervasyonları gösteren ekran

Dosyanın görevi: Bu dosya daha önce oluşturulmuş kayıtların kullanıcıya iki ayrı kategori hâlinde sunulmasını sağlar. ‘SegmentedButton‘ ile yaklaşan ve geçmiş kayıtlar arasında geçiş yapılır.

Satır sayısı: 169

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/hotel_models.dart';
import '../state/hotel_app_state.dart';
import '../utils/formatters.dart';
import '../widgets/hotel_widgets.dart';

enum ReservationFilter { upcoming, past }

class ReservationsScreen extends StatefulWidget {
  const ReservationsScreen({super.key});

  @override
  State<ReservationsScreen> createState() => _ReservationsScreenState();
}

class _ReservationsScreenState extends State<ReservationsScreen> {
  ReservationFilter _filter = ReservationFilter.upcoming;

  @override
  Widget build(BuildContext context) {
    final width = MediaQuery.sizeOf(context).width;
    final padding = width >= 1080 ? 24.0 : 16.0;

    return Consumer<HotelAppState>(
      builder: (context, state, _) {
        final items = _filter == ReservationFilter.upcoming
            ? state.upcomingReservations
            : state.pastReservations;

        return Align(
          alignment: Alignment.topCenter,
          child: ConstrainedBox(
            constraints: const BoxConstraints(maxWidth: 1240),
            child: SingleChildScrollView(
              padding: EdgeInsets.fromLTRB(padding, 18, padding, 28),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  SectionHeader(
                    eyebrow: 'Rezervasyonlar',
                    title: 'şYaklaan ve geçşmi',
                    subtitle: '${state.upcomingReservations.length} şyaklaan • '
                        '${state.pastReservations.length} geçşmi',
                  ),
                  const SizedBox(height: 16),
                  SegmentedButton<ReservationFilter>(
                    selected: {_filter},
                    showSelectedIcon: false,
                    segments: const [
                      ButtonSegment(
                        value: ReservationFilter.upcoming,
                        label: Text('şYaklaan'),
                      ),
                      ButtonSegment(
                        value: ReservationFilter.past,
                        label: Text('Geçşmi'),
                      ),
                    ],
```

```
                    onSelectionChanged: (selection) {
                      setState(() {
                        _filter = selection.first;
                      });
                    },
                  ),
                  const SizedBox(height: 16),
                  if (items.isEmpty)
                    const SoftPanel(
                      child: Text('Bu listede rezervasyon yok.'),
                    )
                  else
                    Column(
                      children: items
                          .map(
                            (reservation) => Padding(
                              padding: const EdgeInsets.only(bottom: 12),
                              child: _ReservationCard(reservation: reservation),
                            ),
                          )
                          .toList(),
                    ),
                ],
              ),
            ),
          ),
        );
      },
    );
  }
}

class _ReservationCard extends StatelessWidget {
  const _ReservationCard({
    required this.reservation,
  });

  final Reservation reservation;

  @override
  Widget build(BuildContext context) {
    final isPast = reservation.checkOut.isBefore(DateTime.now());
    return SoftPanel(
      backgroundColor:
          isPast ? const Color(0xFFF8F3EE) : const Color(0xFFF6FAFB),
      padding: const EdgeInsets.all(16),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Container(
            width: 6,
            height: 104,
            decoration: BoxDecoration(
              color: isPast
                  ? const Color(0xFF9E7A5E)
                  : Theme.of(context).colorScheme.primary,
              borderRadius: BorderRadius.circular(999),
            ),
          ),
          const SizedBox(width: 16),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Row(
                  children: [
```

```
                    Expanded(
                      child: Text(
                        reservation.hotelName,
                        style: Theme.of(context).textTheme.titleLarge,
                      ),
                    ),
                    FeaturePill(label: isPast ? 'Geçşmi' : 'şYaklaan'),
                  ],
                ),
                const SizedBox(height: 6),
                Text(reservation.roomName),
                const SizedBox(height: 10),
                Wrap(
                  spacing: 8,
                  runSpacing: 8,
                  children: [
                    FeaturePill(
                      label: formatDateRange(
                        reservation.checkIn,
                        reservation.checkOut,
                      ),
                    ),
                    FeaturePill(label: '${reservation.guests} misafir'),
                    FeaturePill(label: '${reservation.nights} gece'),
                    FeaturePill(label: formatCurrency(reservation.totalPrice)),
                    ...reservation.extras
                        .map((item) => FeaturePill(label: item)),
                  ],
                ),
                const SizedBox(height: 10),
                Text(
                  reservation.guestName,
                  style: Theme.of(context).textTheme.titleMedium,
                ),
                Text(reservation.email),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```

Açıklama: Burada asıl dikkat çeken nokta, veri listesinin doğrudan ham hâlde değil, state katmanında ayrıştırılmış listeler üzerinden okunmasıdır. Böylece ekran tarafında daha temiz ve sade bir kod yapısı elde edilmiştir.

12. lib/utils/formatters.dart - Para ve tarih biçimlendirme yardımcıları

Dosyanın görevi: Bu dosya ekranda tekrar tekrar kullanılan para ve tarih gösterimlerini tek bir yerde toplar.
Satır sayısı: 37

```
String formatCurrency(num amount) {
  final digits = amount.round().toString();
  final buffer = StringBuffer();

  for (var index = 0; index < digits.length; index++) {
    final reverseIndex = digits.length - index;
    buffer.write(digits[index]);
    if (reverseIndex > 1 && reverseIndex % 3 == 1) {
      buffer.write('.');
    }
  }

  return ₺'$buffer';
}

String formatLongDate(DateTime date) {
  const months = [
    'Ocak',
    Ş'ubat',
    'Mart',
    'Nisan',
    'ıMays',
    'Haziran',
    'Temmuz',
    'ğAustos',
    'Eylül',
    'Ekim',
    'ıKasm',
    'ıAralk',
  ];

  return '${date.day} ${months[date.month - 1]} ${date.year}';
}

String formatDateRange(DateTime start, DateTime end) {
  return '${formatLongDate(start)} - ${formatLongDate(end)}';
}
```

Açıklama: Küçük görünse de proje bütünlüğü için yararlıdır. Biçimlendirme fonksiyonlarının ayrı tutulması, aynı dönüşümün farklı ekranlarda kopyalanmasını engeller.

13. lib/widgets/hotel_widgets.dart - Tekrar kullanılabilir ortak widget bileşenleri

Dosyanın görevi: Bu dosya kart, başlık, etiket, metrik kutusu ve görsel kahraman alanı gibi birçok ortak arayüz parçasını içerir.

Satır sayısı: 318

```
import 'package:flutter/material.dart';

class SoftPanel extends StatelessWidget {
  const SoftPanel({
    super.key,
    required this.child,
    this.padding = const EdgeInsets.all(18),
    this.backgroundColor,
    this.borderColor,
    this.radius = 22,
  });

  final Widget child;
  final EdgeInsetsGeometry padding;
  final Color? backgroundColor;
  final Color? borderColor;
  final double radius;

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    return Container(
      padding: padding,
      decoration: BoxDecoration(
        color: backgroundColor ?? Colors.white,
        borderRadius: BorderRadius.circular(radius),
        border: Border.all(
          color:
              borderColor ?? theme.colorScheme.primary.withValues(alpha: 0.08),
        ),
        boxShadow: [
          BoxShadow(
            color: theme.colorScheme.primary.withValues(alpha: 0.035),
            blurRadius: 18,
            offset: const Offset(0, 8),
          ),
        ],
      ),
      child: child,
    );
  }
}

class SectionHeader extends StatelessWidget {
  const SectionHeader({
    super.key,
    required this.eyebrow,
    required this.title,
    required this.subtitle,
    this.trailing,
  });

  final String eyebrow;
  final String title;
  final String subtitle;
  final Widget? trailing;

  @override
  Widget build(BuildContext context) {
    final textTheme = Theme.of(context).textTheme;
```

```
    return LayoutBuilder(
      builder: (context, constraints) {
        final shouldStack = trailing != null && constraints.maxWidth < 720;
        final content = Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              eyebrow.toUpperCase(),
              style: textTheme.labelLarge?.copyWith(
                letterSpacing: 1.1,
                color: Theme.of(context).colorScheme.secondary,
              ),
            ),
            const SizedBox(height: 8),
            Text(title, style: textTheme.headlineMedium),
            const SizedBox(height: 6),
            Text(subtitle, style: textTheme.bodyLarge),
          ],
        );

        if (shouldStack) {
          return Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              content,
              if (trailing != null) ...[
                const SizedBox(height: 16),
                trailing!,
              ],
            ],
          );
        }

        return Row(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Expanded(child: content),
            if (trailing != null) ...[
              const SizedBox(width: 16),
              trailing!,
            ],
          ],
        );
      },
    );
  }
}

class MetricCard extends StatelessWidget {
  const MetricCard({
    super.key,
    required this.value,
    required this.label,
    required this.detail,
    required this.icon,
  });

  final String value;
  final String label;
  final String detail;
  final IconData icon;

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    return SoftPanel(
```

```
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Icon(icon, color: theme.colorScheme.secondary),
          const SizedBox(height: 20),
          Text(value, style: theme.textTheme.displayMedium),
          const SizedBox(height: 4),
          Text(label, style: theme.textTheme.titleMedium),
          const SizedBox(height: 8),
          Text(detail, style: theme.textTheme.bodyMedium),
        ],
      ),
    );
  }
}

class FeaturePill extends StatelessWidget {
  const FeaturePill({
    super.key,
    required this.label,
    this.icon,
  });

  final String label;
  final IconData? icon;

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 8),
      decoration: BoxDecoration(
        color: theme.colorScheme.primary.withValues(alpha: 0.05),
        borderRadius: BorderRadius.circular(999),
        border: Border.all(
          color: theme.colorScheme.primary.withValues(alpha: 0.08),
        ),
      ),
      child: Row(
        mainAxisSize: MainAxisSize.min,
        children: [
          if (icon != null) ...[
            Icon(icon, size: 16, color: theme.colorScheme.secondary),
            const SizedBox(width: 8),
          ],
          Flexible(
            child: Text(
              label,
              style: theme.textTheme.labelLarge?.copyWith(
                color: theme.colorScheme.primary,
              ),
            ),
          ),
        ],
      ),
    );
  }
}

class PhotoHero extends StatelessWidget {
  const PhotoHero({
    super.key,
    required this.imagePath,
    required this.eyebrow,
    required this.title,
    required this.subtitle,
```

```
    this.height = 260,
    this.action,
  });

  final String imagePath;
  final String eyebrow;
  final String title;
  final String subtitle;
  final double height;
  final Widget? action;

  @override
  Widget build(BuildContext context) {
    final textTheme = Theme.of(context).textTheme;
    return LayoutBuilder(
      builder: (context, constraints) {
        final wide = constraints.maxWidth >= 880;
        final imageHeight = height < 220 ? 168.0 : 184.0;

        final label = Container(
          padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 7),
          decoration: BoxDecoration(
            color: Theme.of(context).colorScheme.secondary.withValues(
                  alpha: 0.12,
                ),
            borderRadius: BorderRadius.circular(999),
          ),
          child: Text(
            eyebrow.toUpperCase(),
            style: textTheme.labelLarge?.copyWith(
              color: Theme.of(context).colorScheme.secondary,
              letterSpacing: 1.0,
            ),
          ),
        );

        final content = Padding(
          padding: const EdgeInsets.all(22),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            mainAxisAlignment:
                wide ? MainAxisAlignment.center : MainAxisAlignment.start,
            children: [
              label,
              const SizedBox(height: 14),
              ConstrainedBox(
                constraints: const BoxConstraints(maxWidth: 520),
                child: Text(
                  title,
                  style: (wide
                          ? textTheme.displayMedium
                          : textTheme.headlineMedium)
                      ?.copyWith(height: 1.05),
                ),
              ),
              const SizedBox(height: 10),
              ConstrainedBox(
                constraints: const BoxConstraints(maxWidth: 520),
                child: Text(subtitle, style: textTheme.bodyLarge),
              ),
              if (action != null) ...[
                const SizedBox(height: 18),
                action!,
              ],
            ],
          ),
```

```
        );

        final image = ClipRRect(
          borderRadius: wide
              ? const BorderRadius.only(
                  topRight: Radius.circular(24),
                  bottomRight: Radius.circular(24),
                )
              : const BorderRadius.only(
                  topLeft: Radius.circular(24),
                  topRight: Radius.circular(24),
                ),
          child: SizedBox(
            height: wide ? double.infinity : imageHeight,
            width: double.infinity,
            child: Image.asset(imagePath, fit: BoxFit.cover),
          ),
        );

        return Container(
          decoration: BoxDecoration(
            color: Colors.white,
            borderRadius: BorderRadius.circular(24),
            border: Border.all(
              color:
                  Theme.of(context).colorScheme.primary.withValues(alpha: 0.08),
            ),
            boxShadow: [
              BoxShadow(
                color: Theme.of(context)
                    .colorScheme
                    .primary
                    .withValues(alpha: 0.03),
                blurRadius: 22,
                offset: const Offset(0, 10),
              ),
            ],
          ),
          child: wide
              ? SizedBox(
                  height: height,
                  child: Row(
                    children: [
                      Expanded(flex: 11, child: content),
                      Expanded(flex: 9, child: image),
                    ],
                  ),
                )
              : Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    image,
                    content,
                  ],
                ),
        );
      },
    );
  }
}
```

Açıklama: Tekrarlı arayüz kodunun ortak widget’lara ayrılması, hem okunabilirliği yükseltmiş hem de ekran dosyalarının sadece kendi iş mantıklarına odaklanmasını sağlamıştır.

14. test/widget_test.dart - Widget testleri ve temel senaryolar

Dosyanın görevi: Bu dosya uygulamanın bazı temel akışlarını test eder. Menülerin görünmesi, boş rezervasyon ekranı, otel seçimi, doğrulama mesajı ve filtre davranışı burada kontrol edilir.

Satır sayısı: 110

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:shared_preferences/shared_preferences.dart';

import 'package:hotel_reservation/app/app.dart';

void main() {
  setUp(() {
    SharedPreferences.setMockInitialValues({});
  });

  testWidgets('renders the commercial navigation', (WidgetTester tester) async {
    await tester.pumpWidget(const MyApp());
    await tester.pumpAndSettle();

    expect(find.text('Oteller'), findsWidgets);
    expect(find.text('Rezervasyon'), findsOneWidget);
    expect(find.text('Rezervasyonlar'), findsOneWidget);
    expect(find.text('Liman şığıI Oteli'), findsOneWidget);
  });

  testWidgets('booking screen is empty before hotel selection',
      (WidgetTester tester) async {
    await tester.pumpWidget(const MyApp());
    await tester.pumpAndSettle();

    await tester.tap(find.text('Rezervasyon').last);
    await tester.pumpAndSettle();

    expect(find.text('Seçili tesis yok'), findsOneWidget);
  });

  testWidgets('hotel tap opens hotel specific booking flow',
      (WidgetTester tester) async {
    await tester.pumpWidget(const MyApp());
    await tester.pumpAndSettle();

    await tester.scrollUntilVisible(
      find.text('ğBoaz ıSaray'),
      300,
      scrollable: find.byType(Scrollable).first,
    );
    await tester.tap(find.text('ğBoaz ıSaray'));
    await tester.pumpAndSettle();

    expect(find.text('Saray Süiti'), findsOneWidget);
    expect(find.text('Deniz Delüks'), findsWidgets);
    expect(find.text('ğBoaz Süiti'), findsWidgets);
    expect(find.text('Rezervasyonu tamamla'), findsOneWidget);
    expect(find.textContaining('ıKahvalt'), findsWidgets);
  });

  testWidgets('shows validation on empty booking form',
      (WidgetTester tester) async {
    await tester.pumpWidget(const MyApp());
    await tester.pumpAndSettle();

    await tester.scrollUntilVisible(
      find.text('Liman şığıI Oteli'),
      200,
```

```
      scrollable: find.byType(Scrollable).first,
    );
    await tester.tap(find.text('Liman şığıI Oteli'));
    await tester.pumpAndSettle();

    await tester.tap(find.text('Rezervasyon').last);
    await tester.pumpAndSettle();

    await tester.scrollUntilVisible(
      find.text('Rezervasyonu tamamla'),
      300,
      scrollable: find.byType(Scrollable).first,
    );
    await tester.tap(find.text('Rezervasyonu tamamla'));
    await tester.pump();

    expect(find.text('Eksik ıalanlar doldurun.'), findsOneWidget);
  });

  testWidgets('opens reservations menu', (WidgetTester tester) async {
    await tester.pumpWidget(const MyApp());
    await tester.pumpAndSettle();

    await tester.tap(find.text('Rezervasyonlar'));
    await tester.pumpAndSettle();

    expect(find.text('şYaklaan'), findsOneWidget);
    expect(find.text('Geçşmi'), findsOneWidget);
  });

  testWidgets('filters hotels by selected feature',
      (WidgetTester tester) async {
    await tester.pumpWidget(const MyApp());
    await tester.pumpAndSettle();

    await tester.tap(find.text('Filtreler'));
    await tester.pumpAndSettle();

    await tester.tap(find.widgetWithText(FilterChip, 'Spa'));
    await tester.pumpAndSettle();

    await tester.ensureVisible(find.text('Uygula'));
    await tester.pumpAndSettle();
    await tester.tap(find.text('Uygula'));
    await tester.pumpAndSettle();

    expect(find.text('Liman şığıI Oteli'), findsOneWidget);
    expect(find.text('ğBoaz ıSaray'), findsNothing);
  });
}
```

Açıklama: Ders projelerinde test bölümü çoğu zaman eksik bırakılır. Bu projede en azından temel kullanıcı senaryolarının otomatik olarak doğrulanması, uygulamanın daha ciddi ve kontrollü hazırlandığını gösterir.

Uygulama Ekran Görselleri

Aşağıda uygulamadan alınan ekran görüntüleri ikili yerleşim ile sunulmuştur.







